# utility-viz 软件包：效用与消费者选择图形

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-03 · v2.0.0b1 ·  · 

## 目录

**1 简介 ..... 2**
1.1 功能范围 ..... 2
1.2 阅读指引 ..... 2
**2 安装 ..... 3**
2.1 系统需求 ..... 3
2.2 安装 uv ..... 3
2.3 安装 utility-viz ..... 3
2.4 可选依赖 ..... 4
2.5 安装命令行工具 ..... 4
2.6 开发环境设置 ..... 4
2.7 验证安装 ..... 4
2.8 从 econ-viz 迁移 ..... 5
**3 快速开始 ..... 6**
3.1 基本示例 ..... 6
3.2 逐步说明 ..... 6
**4 Canvas 画布 ..... 8**
4.1 构造函数 ..... 8
4.2 绘图方法 ..... 9
4.3 样式 ..... 15
4.4 链式调用 ..... 18
**5 多面板图与需求图 ..... 18**
5.1 多面板图 ..... 18
5.2 路径 ..... 20
5.3 需求图 ..... 21
5.4 价格效应分解 ..... 21
5.5 Edgeworth 箱形图 ..... 24
**6 核心模型 ..... 25**
6.1 平滑偏好 ..... 25
6.2 折点与角解 ..... 27
6.3 收入与参考点 ..... 30
**7 进阶模型 ..... 35**
**8 分析 ..... 37**
8.1 比较静态 ..... 37
8.2 Slutsky 矩阵 ..... 38
8.3 齐次性 ..... 38
8.4 规模报酬 ..... 39
**9 主题与设置 ..... 40**
9.1 内置主题 ..... 40
9.2 自定义主题 ..... 44
9.3 配置文件 ..... 45
**10 导出 ..... 49**
10.1 输出格式 ..... 49
10.2 GIF 动画 ..... 49
10.3 交互窗口 ..... 49
**11 命令行工具 ..... 49**
11.1 说明与模型 ..... 50
11.2 绘图 ..... 50
11.3 创建配置文件 ..... 53
11.4 需求公式 ..... 53
**12 动画 ..... 54**
**13 交互组件 ..... 57**
13.1 笔记本设置 ..... 57
13.2 选择交互组件或 GIF ..... 57
**14 LaTeX 解析 ..... 58**
14.1 支持的形式 ..... 58
14.2 错误处理 ..... 58
14.3 在命令行工具中使用 ..... 59
**更新纪录 ..... 61**
**命令索引 ..... 65**
**参考文献 ..... 67**

[*]台湾政治大学国际经营与贸易学系，台北，台湾。
[†]通讯作者。电子邮箱：contact@econ-viz.org

# 第 1 节　简介

utility-viz 是微观经济学绘图软件包，涵盖效用模型、最优消费组合求解与可视化。Python API 与命令行工具均可生成无差异曲线、预算线、消费者均衡、需求曲线及 Edgeworth 箱形图。

图形默认显示第一象限，坐标轴以箭头表示并隐藏数字刻度，标签支持 TeX 数学语法。输出格式包括 PNG、PDF、SVG、TikZ 与 GIF。

本软件包在 1.12 版以前名为 econ-viz，2.0.0 起更名为 utility-viz；变更内容与旧代码的兼容方式详见第 2.8 节。本手册说明预览版 2.0.0b1。

utility-viz 属于 EconViz 系列软件包：绘图通过与领域无关的场景与渲染库 mosaickit，曲线则以 bezierkit 描绘。经济学原理图形由同系列的 principle-viz 负责。

## 1.1　功能范围

utility-viz 的功能分为效用模型、均衡求解、效用水平、图层与导出格式五个部分。模型与求解结果可分开使用；同一模型也可交由画布、多面板图、需求图或分析 API 处理。

## 1.2　阅读指引

本手册按主题分成四个部分（参见表 1）：

**表 1**
章节主题

| 主题 | 子主题 | 说明 | 章节 |
|---|---|---|---|
| 绘图与导出 | 画布与图层 | 坐标轴、曲线与均衡点 | 第 4 节 |
| | 多面板图与需求图 | 并排比较与需求曲线 | 第 5 节 |
| | 主题与配置文件 | 配色、样式与配置文件 | 第 9 节 |
| | 输出格式 | 导出图像与 TikZ | 第 10 节 |
| 模型与分析 | 内置模型 | 常见效用函数 | 第 6 节 |
| | 自定义与多商品模型 | 自定义函数与多商品、场景 | 第 7 节 |
| | 比较静态与 Slutsky 矩阵 | 需求导数与齐次性 | 第 8 节 |
| 动画与交互 | GIF 动画 | 参数变动的 GIF | 第 12 节 |
| | Jupyter 交互组件 | 笔记本滑块调参数 | 第 13 节 |
| 其他输入方式 | 命令行工具 | 使用命令行出图 | 第 11 节 |
| | LaTeX 效用函数解析 | 从算式创建模型 | 第 14 节 |

初次使用时，依次阅读第 3 节与第 4 节。查询特定 API 时，可直接前往对应章节或命令索引。从 econ-viz 1.x 升级时，请先阅读第 2.8 节。

# 第 2 节　安装

## 2.1　系统需求

utility-viz 需要 Python 3.10 以上版本[1]。本章以 uv[2] 管理软件包与项目，并附上对应的 pip 命令。

安装软件包时会一并安装 NumPy、SciPy、matplotlib 与 SymPy，分别用于数组运算、数值求解、绘图与符号运算，以及用于与后端无关的场景与 Bézier 曲线的 mosaickit 与 bezierkit[3]。

## 2.2　安装 uv

本手册的命令以 uv 为准。现有项目仍可使用 pip、pipx 或 Poetry；软件包 API 不受管理工具影响。

依操作系统运行下列安装命令。

**macOS、Linux**

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

**Windows**

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
install.ps1 | iex"
```

## 2.3　安装 utility-viz

创建项目并添加 utility-viz：

```
uv init my-diagrams
cd my-diagrams
uv add utility-viz
```

使用 `uv run` 在项目环境中运行程序：

```
uv run python main.py
```

将第 3 节的基本示例存为项目目录下的 `main.py`，再运行上述命令。`uv add` 记录项目依赖，`uv run` 使用该项目的 Python 环境。安装与运行必须使用同一个环境。

若要固定本手册使用的版本，可在添加依赖时指定版本号：

```
uv add "utility-viz==2.0.0b1"
```

2.0.0b1 是预览版。在没有 2.x 正式版之前，`uv add utility-viz` 与 `pip install utility-viz` 会直接安装此版本；正式版发布后，需以 `uv add --prerelease allow utility-viz` 或 `pip install --pre utility-viz` 才能安装预览版。

若使用现有的 Python 虚拟环境，可通过 pip 安装：

```
python -m pip install -U utility-viz
```

---

[1] Python 官方网站提供各操作系统的安装程序：`https://www.python.org/downloads/`。

[2] uv 是 Astral 开发的 Python 软件包与项目管理工具，速度快且可一并管理 Python 版本；安装方式与完整说明见官方文档：`https://docs.astral.sh/uv/`。

[3] 一般 Python 绘图不需要另外安装 LaTeX；只有要编译导出的 TikZ 源代码时，才需要 LaTeX 环境。

软件包名称是 utility-viz，Python 导入名称是 utility_viz：

```
from utility_viz import Canvas, solve
```

import 语句不得使用连字符。若发生 ModuleNotFoundError，检查运行程序的解释器是否与安装软件包时使用的环境相同。

## 2.4　可选依赖

动画与 Jupyter 交互组件需要可选依赖。按用途安装其中一组：

| | |
|---|---|
| animation | 供 Animator 写入 GIF（详见第 12 节）；安装 Pillow。 |
| interactive | 供 WidgetViewer 在 Jupyter 显示控件（详见第 13 节）；安装 ipywidgets 与 IPython。 |
| all | 全部可选依赖。 |

```
uv add "utility-viz[animation]"    # GIF 导出（Pillow）
uv add "utility-viz[interactive]"  # 笔记本交互组件
uv add "utility-viz[all]"          # 全部可选依赖
```

安装可选依赖不会自动运行动画或打开笔记本，仍需按各章示例调用对应的 API。

## 2.5　安装命令行工具

仅使用命令行接口时，可将 utility-viz 安装为独立工具（详见第 11 节）：

```
uv tool install utility-viz
```

此方式将命令行工具安装在独立环境。需要在 Python 程序中导入软件包时，仍须在该项目运行 uv add utility-viz。在项目内以 uv run utility-viz 调用工具，可让命令行与 Python 程序使用同一版本。

## 2.6　开发环境设置

```
git clone https://github.com/EconViz/utility-viz.git
cd utility-viz
uv sync --all-packages --all-extras
```

此仓库是 uv workspace，包含两个同步发布的软件包：根目录的 utility-viz，以及 packages/econ-viz/ 中的 econ-viz，即第 2.8 节所述的兼容包。uv sync --all-packages --all-extras 会以可编辑模式安装两者，并安装开发依赖与所有可选依赖。完成后运行完整的质量检查：

```
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy .
```

## 2.7　验证安装

```
uv run utility-viz --version   # utility-viz 2.0.0b1
uv run utility-viz help
```

注释中的版本号仅为示例，实际输出取决于安装版本。也可用以下命令确认 Python 能导入绘图与求解接口：

```
uv run python -c "from utility_viz import Canvas, solve; print('OK')"
```

在服务器或其他没有图形界面的环境中，请以 save() 或命令行的 --output 输出文件。show() 需要可用的交互式绘图后端，窗口打不开不代表安装失败。

## 2.8　从 econ-viz 迁移

econ-viz 于 2.0.0 更名为 utility-viz（参见表 2）。

**表 2**
1.x 与 2.x 的名称

| | 1.x | 2.x |
|---|---|---|
| 发行包 | `pip install econ-viz` | `pip install utility-viz` |
| 导入 | `import econ_viz` | `import utility_viz` |
| 命令 | `econ-viz` | `utility-viz` |
| 配置文件 | `econ-viz.toml` | `utility-viz.toml` |

**应安装哪个软件包**

utility-viz 只提供 `utility_viz` 包与 `utility-viz` 命令。版本号相同的 econ-viz 2.x 是精简的兼容包：它依赖同版本的 utility-viz，并另外提供 `econ_viz` 包与 `econ-viz` 命令。因此以 `pip install --upgrade econ-viz` 升级 1.x 安装（预览版需加 `--pre`）会改用 2.x，旧代码仍可运行。代码不再导入 `econ_viz` 后，再改为 `pip install utility-viz`。

**兼容层**

在整个 2.x 期间，按文档编写的 1.x 代码仍可运行：

- `import econ_viz` 在每个进程中发出一次弃用警告。
- `from econ_viz import ...` 与文档记载的子模块（`econ_viz.models`、`econ_viz.optimizer`、`econ_viz.themes` 等）会对应到 `utility_viz` 的同名对象；名称未变更者是同一个对象。
- 创建 `econ_viz.Canvas`、`econ_viz.Figure` 或 `econ_viz.animation.Animator`，或访问 `econ_viz.Layout` 时，会发出指明替代名称的 `UtilityVizDeprecationWarning`。
- `econ-viz` 命令会打印弃用警告，再以相同参数运行 `utility-viz`。
- 旧的 `econ-viz.toml` 仍可读取，但会发出警告（详见第 9.3.3 节）；`utility-viz init --migrate` 可将其复制为 `utility-viz.toml`（详见第 11.3 节）。

`UtilityVizDeprecationWarning`

`FutureWarning` 的子类，默认会显示。消息列出弃用版本、移除版本与替代方式，例如：

```
econ_viz.Canvas is deprecated since 2.0.0 and will be removed in 3.0.0;
use utility_viz.Canvas instead.
```

新增：*v2.0.0b1*

**更新代码**

多数代码只需更改导入名称。1.x 中可单独导入的模块已移至别处（参见表 3），其中的名称也可从软件包根目录导入。

**表 3**
已迁移的模块

| 1.x | 2.x |
|---|---|
| `econ_viz.optimizer` | `utility_viz`（根目录）或 `utility_viz.models` |
| `econ_viz.analysis` | `utility_viz.models` |
| `econ_viz.exceptions` | `utility_viz`（根目录） |
| `econ_viz.animation` | `utility_viz.core.animation` |

| 1.x | 2.x |
|---|---|
| `econ_viz.interactive` | `utility_viz.core.interactive` |

公开 API 为软件包根目录（`from utility_viz import Canvas, ...`）与 `utility_viz.models`。其他 `utility_viz.core.*` 模块属高级 API，可能在次要版本间迁移。

**3.0.0 的移除范围**

`econ_viz` 包、`econ-viz` 命令与 `econ-viz.toml` 查找于 3.0.0 移除，而非 2.0.0。兼容层只涵盖文档记载的 1.x API；未记载的深层路径（例如 `econ_viz.canvas.renderers`）仅尽力对应，随时可能移除。

# 第 3 节　快速开始

## 3.1　基本示例

本章以 Cobb-Douglas 消费者问题说明完整流程：

$$\max_{x,y} u(x,y) = x^{\frac{1}{2}} y^{\frac{1}{2}}$$

$$\text{s.t. } 2x + 3y = 30$$

程序先求解最优消费组合，再绘制无差异曲线、预算集与均衡点（参见图 1）。

**例 1**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
lvls = levels.around(eq.utility, n=5)  # 五条曲线 一条过最优点
cvs.add_utility(model, levels=lvls)
cvs.add_budget(2.0, 3.0, 30.0, fill=True)  # 预算线与可行集合
cvs.add_equilibrium(eq, show_ray=True)  # 附扩张路径
cvs.save("cobb_douglas.png")
```

运行主程序 `main.py` 后，会在当前工作目录创建 `cobb_douglas.png`。本例的最优消费量为 $x^* = 7.5$、$y^* = 5$，支出为 $2 \times 7.5 + 3 \times 5 = 30$，恰好等于收入。预算线的两个轴截距则为 $\frac{I}{p_x} = 15$ 与 $\frac{I}{p_y} = 10$。

## 3.2　逐步说明

绘图流程依次为：创建模型、求解均衡、选取效用水平、创建画布、添加图层与导出。

`utility_viz.models`

```python
from utility_viz.models import CobbDouglas
model = CobbDouglas(alpha=0.5, beta=0.5)
```

创建效用模型。内置模型详见第 6 节；自定义函数使用 `CustomUtility`，详见第 7 节。

`solve`

```
solve(⟨模型⟩, px=⟨浮点数⟩, py=⟨浮点数⟩, income=⟨浮点数⟩)
```

根据价格与收入求解，返回不可变的 `Equilibrium` 对象。

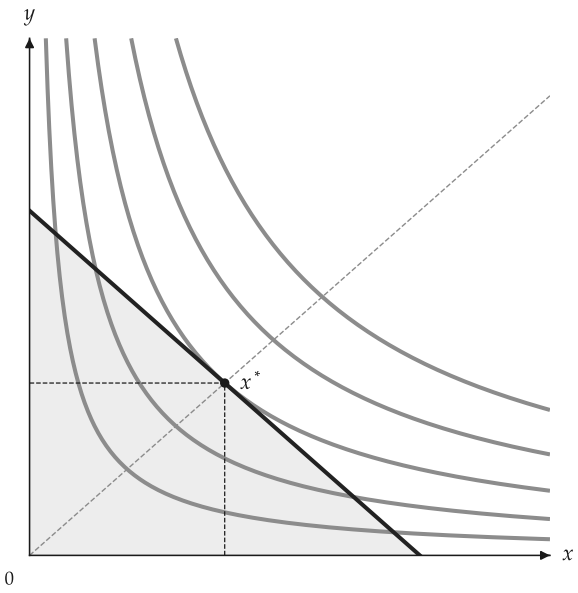



**图 1**
消费者均衡。

`x` *float*
最优消费量 $x^*$。

`y` *float*
最优消费量 $y^*$。

`utility` *float*
模型在最优点的效用值。

`bundle_type` *str*
解的类型：`"interior"` 为内部解、`"boundary"` 为数值解位于数量下界（例如 Stone-Geary 的最低消费限制）、`"kink"` 为完全互补的折点解、`"corner"` 为完全替代的角解。

```python
from utility_viz import solve
eq = solve(model, px=2.0, py=3.0, income=30.0)
print(eq.x, eq.y, round(eq.utility, 3))

# 7.5 5.0 6.124
```

`levels.around`
`levels.around(⟨效用值⟩, n=⟨整数⟩)`
以指定效用值为中心，生成 $n$ 个效用水平。图 1 使用 `n=5`，因此绘制五条无差异曲线。

`levels.around()` 会保留指定的效用值，均衡点会落在其中一条无差异曲线上。若直接传入 `levels=5`，画布会依采样到的效用范围选取水平，不保证包含均衡效用。要比较多张图上的同一组偏好，可先计算一次 `lvls`，再让各张图使用相同水平。

`Canvas`
`Canvas(x_max=⟨浮点数⟩, y_max=⟨浮点数⟩, ...)`
创建画布并设置坐标范围，其余参数详见第 4 节。

坐标范围只控制显示区域，不参与 `solve()` 的计算。均衡点未出现在图中时，检查 `eq.x` 与 `eq.y` 是否超出范围，再调整坐标上限。

`Canvas.add_*`
在同一张画布上添加无差异曲线、预算线或均衡点。这些方法可以链式调用（详见第 4.4 节）。`add_equilibrium()` 使用现有的求解结果，不会重新求解；若之后修改价格、收入或模型，需重新调用 `solve()`，并创建对应的新图。

Canvas.save
Canvas.show

save() 根据扩展名导出文件（详见第 10 节）；show() 打开 matplotlib 交互窗口。输出路径以运行程序时的工作目录为准，若需指定子目录，请事先创建目录。

**检查输入与求解失败**

价格与收入必须为正；输入不符合条件时，solve() 抛出 InvalidParameterError[4]。

求解未收敛时，solve() 抛出 OptimizationError，不会返回无效的均衡点。

```
from utility_viz import InvalidParameterError, OptimizationError, solve

try:
    eq = solve(model, px=2.0, py=3.0, income=30.0)
except (InvalidParameterError, OptimizationError) as error:
    print(error)
else:
    print(round(eq.x, 3), round(eq.y, 3), eq.bundle_type)
```

软件包抛出的所有异常都继承自 UtilityVizError，捕获它即可一并处理。1.x 的名称 EconVizError 是同一个类。

数值解可能有微小误差，例如结果接近但不恰好等于 7.5。验证时应使用适当容差；显示结果时再用 round() 格式化，避免过早四舍五入影响后续计算。

# 第 4 节　Canvas 画布

Canvas 以 matplotlib 的 Figure 与 Axes 为基础，提供无差异曲线、预算线与均衡点等图层。

## 4.1　构造函数

Canvas

```
from utility_viz import ArrowStyle, Canvas, Stroke, themes

cvs = Canvas(
    x_max=20,
    y_max=15,
    x_label="x",
    y_label="y",
    title=r"Cobb-Douglas $x^{0.5} y^{0.5}$",
    dpi=300,
    x_label_pos="right",    # top / right / bottom
    y_label_pos="top",      # left / top / right
    font="DejaVu Sans",
    math_font="stix",
    axis_stroke=Stroke(width=1.0, arrow=ArrowStyle.TRIANGLE),
    theme=themes.default,
)
```

更新：*v1.7.0*

创建画布，并设置坐标范围、轴标签、字体、线条样式与主题。未明确传入的视觉设置取自当前的 Config 与 Theme。

x_max
*float* · 默认 10

横轴上限。

y_max
*float* · 默认 10

纵轴上限。

[4] 一般平滑模型使用序列最小二乘法（sequential least squares programming, SLSQP）求解。

`x_label`　横轴末端的标签。

*str* · 默认 `"X"`

`y_label`　纵轴末端的标签。

*str* · 默认 `"Y"`

`title`　图形标题。

*str* | *None* · 默认 `None`

`dpi`　位图导出分辨率，限制在 1–1200 之间。

*int* · 默认 `300`

`x_label_pos`　横轴标签相对箭头的位置，也接受 `LabelPosition`（参见表 4）。

*str* · 默认 `"right"`

**表 4**　`x_label_pos` 值

| 值 | 位置 |
|---|---|
| `"top"` | 箭头上方 |
| `"right"` | 箭头右侧 |
| `"bottom"` | 箭头下方 |

`y_label_pos`　纵轴标签相对箭头的位置（参见表 5）。

*str* · 默认 `"top"`

**表 5**　`y_label_pos` 值

| 值 | 位置 |
|---|---|
| `"left"` | 箭头左侧 |
| `"top"` | 箭头上方 |
| `"right"` | 箭头右侧 |

`font`　所有文字使用的字体，或候选字体列表。此项设置仅作用于指定画布，不会改变 matplotlib 的全局设置。

*str* | *list* · 默认 `None`

`math_font`　matplotlib 的数学字体：`dejavusans`、`dejavuserif`、`cm`、`stix` 或 `stixsans` 等。

*str* · 默认 `None`

`axis_stroke`　同时设置两轴的粗细、线条样式、颜色与箭头（详见第 4.3 节）。

*Stroke* · 默认 `theme`

`x_axis_stroke`　单独覆盖横轴。

*Stroke* · 默认 `None`

`y_axis_stroke`　单独覆盖纵轴。

*Stroke* · 默认 `None`

`theme`　配色与样式主题（详见第 9 节）。

*Theme* · 默认 `themes.default`

## 4.2　绘图方法

`add_*` 方法在当前的画布添加图层，并返回同一个 `Canvas`，因此可链式调用（详见第 4.4 节）。

`add_utility`

```
cvs.add_utility(
    func,
```

```
    levels=3,           # 曲线条数或效用值列表
    color=None,         # 默认 theme.ic_color
    linewidth=None,     # 默认 theme.ic_linewidth
    show_rays=False,
    show_kinks=False,
    kink_radius=1.0,
    show_bliss=True,    # 标出极乐点
    show_ic_labels=False,
    stroke=None,
    ray_stroke=None,
    ic_label=None,
    highlight_level=None,
    secondary_stroke=None,
    label_style="numeric",
)
```

绘制无差异曲线。

`levels` — *int | list* · 默认 `3`

曲线条数或效用值列表；例如 `levels.around(eq.utility, n=5)` 会在均衡效用附近选取五个水平。

`highlight_level` — *float | None* · 默认 `None`

焦点效用值。绘制时选择最接近的曲线，以主要样式显示。

`secondary_stroke` — *Stroke | None* · 默认 `None`

其他曲线的样式；省略时使用 `theme.secondary_ic_stroke`。

`show_ic_labels` — *bool* · 默认 `False`

显示曲线标签。标签沿曲线切线旋转，并保持在可见绘图区内。

`label_style` — *str* · 默认 `"numeric"`

`"numeric"` 显示效用值；`"ordinal"` 按效用递增顺序显示 $u_1, u_2, \ldots$。

```
lvls = levels.around(eq.utility, n=5)
cvs.add_utility(
    model,
    levels=lvls,
    highlight_level=eq.utility,
    show_ic_labels=True,
    label_style="ordinal",
)
```

**例 2**

```
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))  # 围绕最优点的三条曲线
cvs.save("add_utility.png")
```

### add_budget

```
cvs.add_budget(
    px, py, income,
    color=None,
    linewidth=None,
    linestyle="-",
    label=None,         # 图例标签 LaTeX
    fill=False,         # 可行集合阴影
    fill_alpha=None,    # 默认 theme.budget_fill_alpha
    stroke=None,
)
```

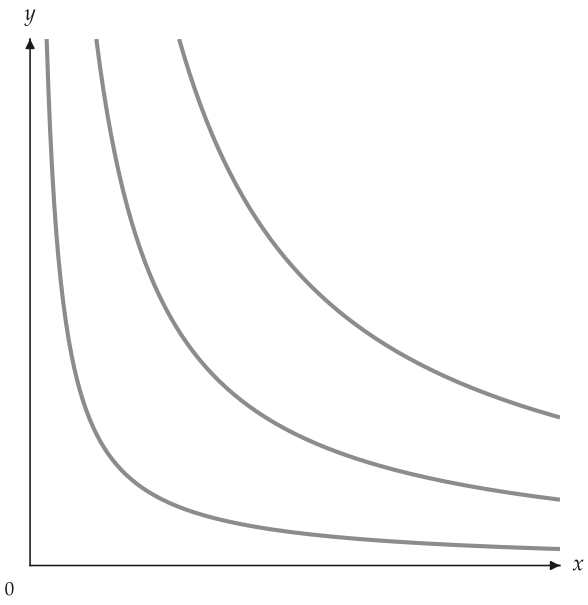



**图 2**
三条无差异曲线。

绘制预算线 $p_x x + p_y y = I$。

`fill`
*bool | Fill* · 默认 `False`
`True` 使用主题的阴影设置；传入 `Fill` 可覆盖阴影颜色与透明度。

`stroke`
*Stroke | None* · 默认 `None`
统一设置预算线的颜色、线宽、线型与透明度，优先于 `color`、`linewidth`、`linestyle` 等简写参数（详见第 4.3 节）。

`label`
*str | None* · 默认 `None`
图例文字；调用 `show_legend()` 后才会显示图例。

**例 3**

```python
from utility_viz import Canvas, Config

Config.load("manual.toml").use()  # 本手册图形所用的设置

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_budget(2.0, 3.0, 30.0, fill=True)  # 预算线与可行集合
cvs.save("add_budget.png")
```

`add_equilibrium`

```python
cvs.add_equilibrium(
    eq,                 # solve() 的返回值
    color=None,
    markersize=None,
    label="x^*",
    drop_dashes=True,   # 到两轴的虚线
    show_ray=False,     # 扩张路径
    drop_stroke=None,
    ray_stroke=None,
    marker=None,
)
```

标记 `solve()` 返回的最优消费组合。

`drop_dashes`
*bool* · 默认 `True`
显示至两轴的投影线。

`show_ray`
*bool* · 默认 `False`
显示通过原点与均衡点的射线。

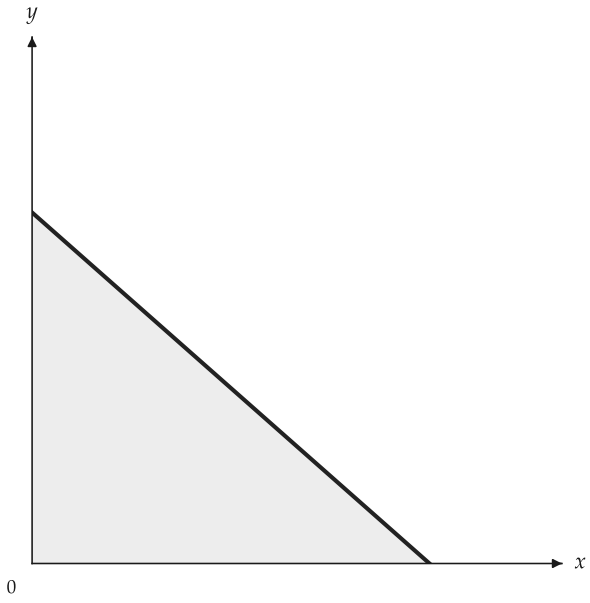



**图 3**
预算线与阴影。

`marker`　均衡点的标记样式（详见第 4.3 节）。

*Marker* | *None* · 默认 `None`

**例 4**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)  # 最优点与通往坐标轴的虚线
cvs.save("add_equilibrium.png")
```

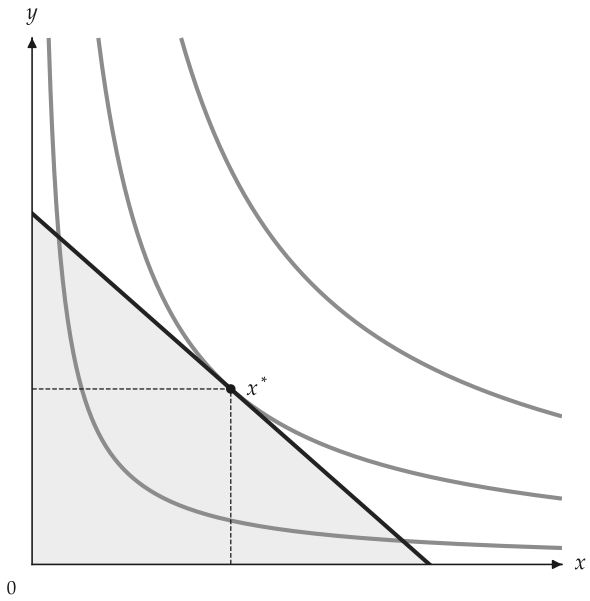



**图 4**
均衡点与虚线。

### add_ray

```
cvs.add_ray(
    slope,              # dy/dx
    color=None,
    linewidth=None,
    stroke=None,
)
```

绘制通过原点、斜率为 `slope`（$\mathrm{d}y/\mathrm{d}x$）的射线，用于标记扩张路径或固定比例。

**例 5**

```
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.add_ray(eq.y / eq.x)  # 通过原点与最优点的射线
cvs.save("add_ray.png")
```

### add_point

```
cvs.add_point(
    x, y,
    label=None,
    color=None,
    markersize=None,
    offset=None,        # 标签位移 pt
    marker=None,
)
```

在指定坐标标记点。

**label**　标记文字，可传入字符串或 `Label`。

*str* | *Label* | *None* · 默认 `None`

**offset**　文字相对标记点的位移，单位为 pt。

*tuple* | *None* · 默认 `None`

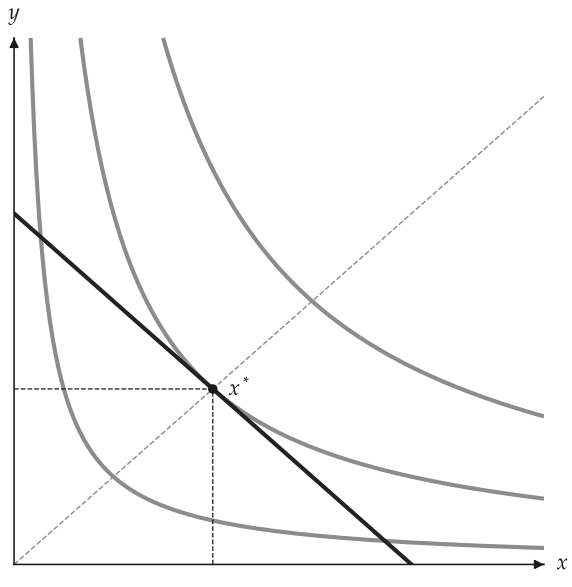



**图 5**
扩张路径射线。

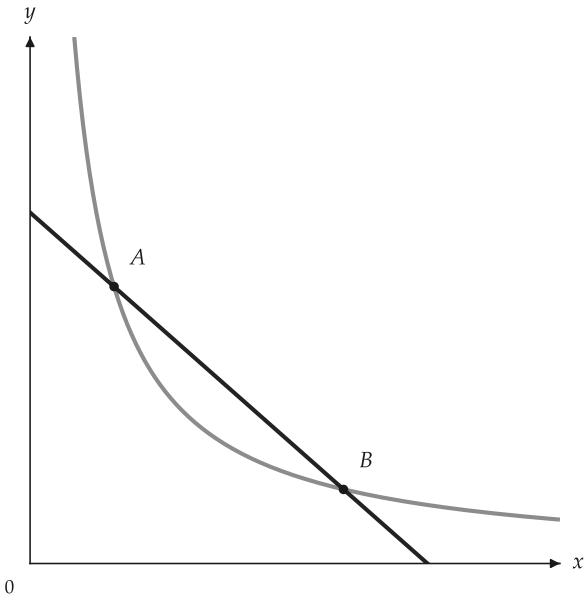



**图 6**
一条无差异曲线与预算线交于两点 $A$ 与 $B$。

`marker`
*Marker* | *None* · 默认 `None`

标记样式。

**例 6**

```python
from utility_viz import Canvas, Config, Label, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_utility(model, levels=[5.0])  # 一条无差异曲线 u = 5
cvs.add_budget(2.0, 3.0, 30.0)
# 与预算线交于两点
# 以 Label 放在点旁
cvs.add_point(3.17, 7.89, label=Label(text="A", position="top-right"))
cvs.add_point(11.83, 2.11, label=Label(text="B", position="top-right"))
cvs.save("add_point.png")
```

`show`
`save`

```python
cvs.show()                # 交互窗口
cvs.save("figure.png")    # .png / .pdf / .svg / .tex
```

`show()` 打开交互窗口；`save()` 根据扩展名导出文件（详见第 10 节），并释放对应的 matplotlib 图形，因此应最后调用。

**例 7**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")  # 斜体的 x 与 y
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq, show_ray=True)  # 均衡点与扩张路径
cvs.save("show_save.png")
```

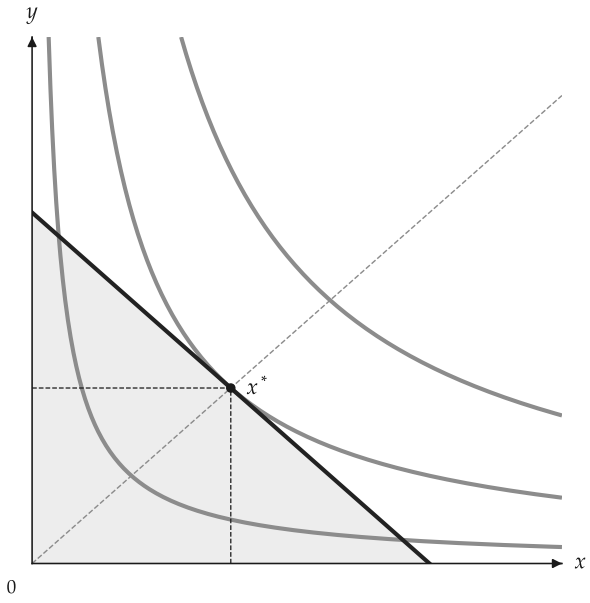



**图 7**
完整画布。

## 4.3　样式

`LineStyle` 定义线型，`ArrowStyle` 定义箭头样式，`Stroke` 统一设置线宽、线型、颜色与箭头，可应用于坐标轴、预算线、路径与投影线。

### 线条样式

`LineStyle`　`LineStyle.SOLID`、`DASHED`、`DOTTED`、`DASHDOT`
新增：*v1.7.0*
提供实线、虚线、点线与点画线（参见图 8）。坐标轴接受枚举值或对应字符串；其他线条通过 `Stroke(style=...)` 设置。`LineStyle` 就是 MosaicKit 的 `DashStyle`，沿用旧名称。

`SOLID`　实线。

*`LineStyle`*
`DASHED`　虚线。

*`LineStyle`*
`DOTTED`　点线。

*`LineStyle`*
`DASHDOT`　点划线。

*`LineStyle`*

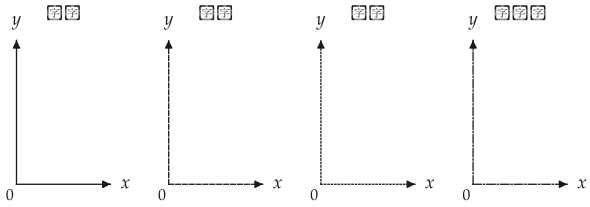



**图 8**
四种线条样式。

**例 8**

```python
import matplotlib.pyplot as plt

from utility_viz import Canvas, Config, LineStyle

Config.load("manual.toml").use()  # 本手册图形所用的设置

styles = [LineStyle.SOLID, LineStyle.DASHED, LineStyle.DOTTED, LineStyle.DASHDOT]
titles = ["实线", "虚线", "点线", "点划线"]

fig, axes = plt.subplots(1, 4, figsize=(7.6, 2.0))
for ax, style, title in zip(axes, styles, titles):
    Canvas(
        title=title,
        x_label="$x$",
        y_label="$y$",
        x_line_style=style,
        y_line_style=style,
        font="PingFang SC",  # 使用有中文字形的字体
        fig=fig,
        ax=ax,
    )

# 预留轴标签的空间
fig.subplots_adjust(wspace=0.62)
fig.savefig("line_styles.png", dpi=160)
```

### 箭头样式

`ArrowStyle`　`ArrowStyle.SIMPLE`、`TRIANGLE`、`FANCY`、`WEDGE`

新增：*v1.7.0*

提供四种箭头样式（参见图 9）。坐标轴通过 `x_arrow_style`、`y_arrow_style` 设置；其他线条通过 `Stroke(arrow=...)` 设置。`ArrowStyle` 是 MosaicKit 的；`SIMPLE` 即 `OPEN`，仍接受 `"->"` 与 `"-|>"`。

`SIMPLE`　开口箭头（`->`）。

*ArrowStyle*

`TRIANGLE`　实心三角形（`-|>`），默认值。

*ArrowStyle*

`FANCY`　弯曲、渐细的箭头。

*ArrowStyle*

`WEDGE`　细楔形箭头。

*ArrowStyle*

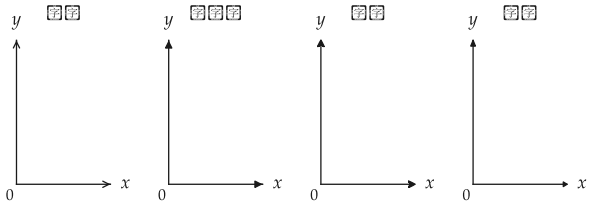



**图 9**
四种箭头样式。

**例 9**

```python
import matplotlib.pyplot as plt

from utility_viz import ArrowStyle, Canvas, Config

Config.load("manual.toml").use()  # 本手册图形所用的设置

styles = [ArrowStyle.SIMPLE, ArrowStyle.TRIANGLE, ArrowStyle.FANCY, ArrowStyle.WEDGE]
titles = ["简单", "三角形", "花式", "楔形"]

fig, axes = plt.subplots(1, 4, figsize=(7.6, 2.0))
for ax, style, title in zip(axes, styles, titles):
    Canvas(
        title=title,
        x_label="$x$",
        y_label="$y$",
        x_arrow_style=style,
        y_arrow_style=style,
        font="PingFang SC",  # 使用有中文字形的字体
        fig=fig,
        ax=ax,
    )

# 预留轴标签的空间
fig.subplots_adjust(wspace=0.62)
fig.savefig("arrow_styles.png", dpi=160)
```

### Stroke

Stroke　`Stroke(width=⟨浮点数⟩, style=⟨线条样式⟩, color=⟨字符串⟩, arrow=⟨箭头样式⟩, opacity=⟨浮点数⟩)`

新增：*v1.7.0*
更新：*v1.9.0*

设置线宽、线型、颜色、箭头与透明度。未指定的字段沿用当前主题。`opacity` 的有效范围为 0 至 1。

### Axis

Axis　`Axis(label=None, label_position=None, stroke=None)`

新增：*v1.8.0*

集中设置单一坐标轴的标签、标签位置与线条。`Canvas`、`Figure`、`DemandDiagram` 与 `EdgeworthBox` 均接受 `x_axis` 与 `y_axis`。若同时传入简写参数，`Axis` 中已设置的字段优先。

```python
from utility_viz import Axis, Canvas, Label, Stroke

cvs = Canvas(
    x_axis=Axis(
        label=Label(text="x_1", fontsize=16),
        label_position="bottom",
        stroke=Stroke(width=1.2),
    ),
    y_axis=Axis(label="x_2"),
)
```

### Marker 与 Label

Marker　`Marker(color=None, size=None, shape=None, opacity=None)`

新增：*v1.8.0*
更新：*v1.9.0*

设置标记点的颜色、大小、形状与透明度。`shape` 接受 matplotlib 的标记字符串，例如 `"o"`、`"s"`、`"^"`、`"D"` 与 `"*"`。

`Label` `Label(text=None, position=None, offset=None, color=None, fontsize=None, visible=None, opacity=None)`

新增：*v1.8.0*
更新：*v1.9.0*

设置文字、位置、位移、颜色、字号、可见性与透明度。位置可为上、下、左、右或四个角落；`Label(visible=False)` 隐藏对应文字。

```python
from utility_viz import Label, Marker

cvs.add_equilibrium(
    eq,
    marker=Marker(shape="s", size=8),
    label=Label(position="bottom-left", offset=8),
)
cvs.add_point(
    12, 2,
    marker=Marker(color="black", shape="D"),
    label=Label(text="A", position="left", fontsize=14),
)
```

**Fill**

`Fill` `Fill(color=None, opacity=None)`

新增：*v1.8.0*
更新：*v1.9.0*

设置阴影区域的颜色与透明度。`alpha` 是 `opacity` 的兼容简写；同一次调用不得传入不同的 `alpha` 与 `opacity`。

```python
from utility_viz import Fill

cvs.add_budget(
    2, 3, 30,
    fill=Fill(color="lightgrey", opacity=0.4),
)
```

### 4.4　链式调用

`add_*` 方法支持链式调用，最后以 `save()` 导出：

```python
Canvas(x_max=20, y_max=15) \
    .add_utility(model, levels=lvls) \
    .add_budget(2.0, 3.0, 30.0, fill=True) \
    .add_equilibrium(eq, show_ray=True) \
    .save("figure.png")
```

## 第 5 节　多面板图与需求图

### 5.1　多面板图

`Figure` `Figure(⟨版面⟩, x_max=⟨浮点数⟩, y_max=⟨浮点数⟩, ..., shared_x=False, shared_y=False)`

新增：*v1.2.0*

创建多面板图，坐标范围与样式参数与 `Canvas` 相同。使用 `fig[idx]` 获取面板，再调用 `add_utility()`、`add_budget()` 等绘图方法。

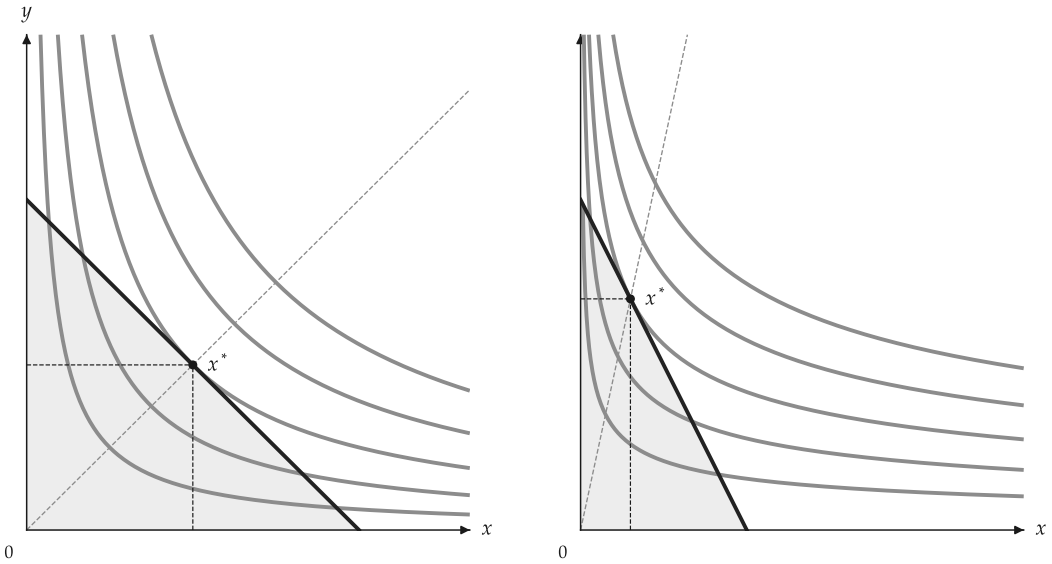



**图 10**
价格变动前后比较。

**例 10**

```python
from utility_viz import Config, Figure, Layout, levels, solve
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

fig = Figure(Layout.SIDE_BY_SIDE, x_max=20, y_max=15, x_label="$x$", y_label="$y$",
shared_y=True)

# 价格上涨前后
cases = [
    (CobbDouglas(alpha=0.5, beta=0.5), 2.0, 3.0, 30.0),
    (CobbDouglas(alpha=0.3, beta=0.7), 4.0, 3.0, 30.0),
]
for idx, (model, px, py, income) in enumerate(cases):
    eq = solve(model, px=px, py=py, income=income)
    panel = fig[idx]  # 面板都是 Canvas
    panel.add_utility(model, levels=levels.around(eq.utility, n=5))
    panel.add_budget(px, py, income, fill=True)
    panel.add_equilibrium(eq, show_ray=True)
fig.save("side_by_side.png")
```

`Layout`
新增：*v1.2.0*
内置版面有 `SINGLE`、`STACKED`、`SIDE_BY_SIDE`、`TOP_TWO_BOTTOM_ONE`、`TOP_ONE_BOTTOM_TWO`、`GRID_2X2` 和 `GRID_3X3`。

`SINGLE`
单个面板。

*Layout*
`STACKED`
上下两个面板。

*Layout*
`SIDE_BY_SIDE`
左右两个面板。

*Layout*
`TOP_TWO_BOTTOM_ONE`
上方两个、下方一个面板。

*Layout*
`TOP_ONE_BOTTOM_TWO`
上方一个、下方两个面板。

*Layout*
`GRID_2X2`
$2 \times 2$ 格四个面板。

*Layout*

`GRID_3X3` 3 × 3 格九个面板。

*Layout*

## 5.2 路径

`PricePath`
`IncomePath`

```python
from utility_viz import IncomePath, LinearBudget, PricePath
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
budget = LinearBudget(px=2.0, py=2.0, income=40.0)

price_path = PricePath(
    model,
    budget=budget,
    price="px",
    price_range=(0.8, 6.0),
    n=40,
)
income_path = IncomePath(
    model,
    budget=budget,
    income_range=(20.0, 80.0),
    n=30,
)
```

新增：*v1.2.0* `PricePath` 与 `IncomePath` 分别计算价格、收入变动时的最优消费组合。使用 `Canvas.add_path()` 绘制价格消费曲线（PCC）或收入消费曲线（ICC）；`PricePath` 也可用于需求图。

`Canvas.add_path`

```python
from utility_viz import Canvas, levels

eq = price_path.equilibria[len(price_path.equilibria) // 2]
lvls = levels.around(eq.utility, n=5)

Canvas(x_max=25, y_max=20) \
    .add_utility(model, levels=lvls) \
    .add_path(price_path, label="PCC") \
    .save("price_path.png")
```

新增：*v1.2.0* 将 `PricePath` 或 `IncomePath` 添加画布。图 11 中，Cobb-Douglas 模型在 $p_x$ 变动时，$y$ 的需求量不变，因此 PCC 为水平线。

**例 11**

```python
from utility_viz import Canvas, Config, LinearBudget, PricePath, levels
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
budget = LinearBudget(px=2.0, py=2.0, income=40.0)
path = PricePath(model, budget=budget, price="px", price_range=(0.8, 6.0), n=40)

# 围绕路径中段最优点的曲线
eq = path.equilibria[len(path.equilibria) // 2]

cvs = Canvas(x_max=25, y_max=20, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_path(path)  # 价格消费曲线
cvs.save("price_path.png")
```

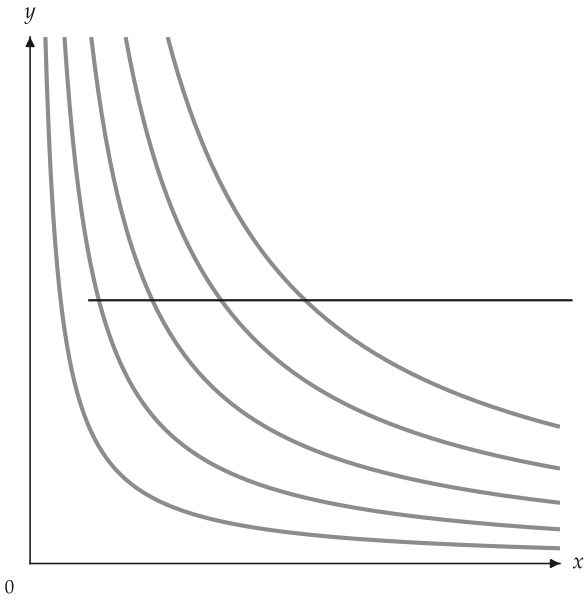



**图 11**
价格消费曲线。

## 5.3　需求图

`DemandDiagram`

```
DemandDiagram(⟨价格路径⟩, title=None, ...)
.add_marshallian_panel(price_markers=None, show_pcc=False,
show_demand_guides=True)
```

新增：*v1.2.0*

根据 `PricePath` 绘制两个面板：上方为预算线与最优消费组合，下方为对应的 Marshall 需求曲线（参见图 12）。目前只接受 `PricePath`，并分别处理平滑、折点与角解。`show_pcc=True` 在上方面板显示价格消费曲线。

**例 12**

```python
from utility_viz import DemandDiagram, LinearBudget, PricePath
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
budget = LinearBudget(px=2.0, py=2.0, income=40.0)
path = PricePath(
    model,
    budget=budget,
    price="px",
    price_range=(0.8, 6.0),
    n=40,
)

fig = DemandDiagram(path, title="Demand: Cobb-Douglas")
fig.add_marshallian_panel(
    price_markers=[1.5, 4.0],
    show_pcc=False,
    show_demand_guides=True,
)
fig.save("demand_cobb_douglas.png")
```

下图由 MosaicKit 图层画成，标签可逐一手动摆放。

## 5.4　价格效应分解

`decompose_price_effect`

```
decompose_price_effect(⟨模型⟩, px=(⟨旧值⟩, ⟨新值⟩), py=⟨浮点数⟩, income=⟨浮点数⟩, method=⟨方法⟩)
```

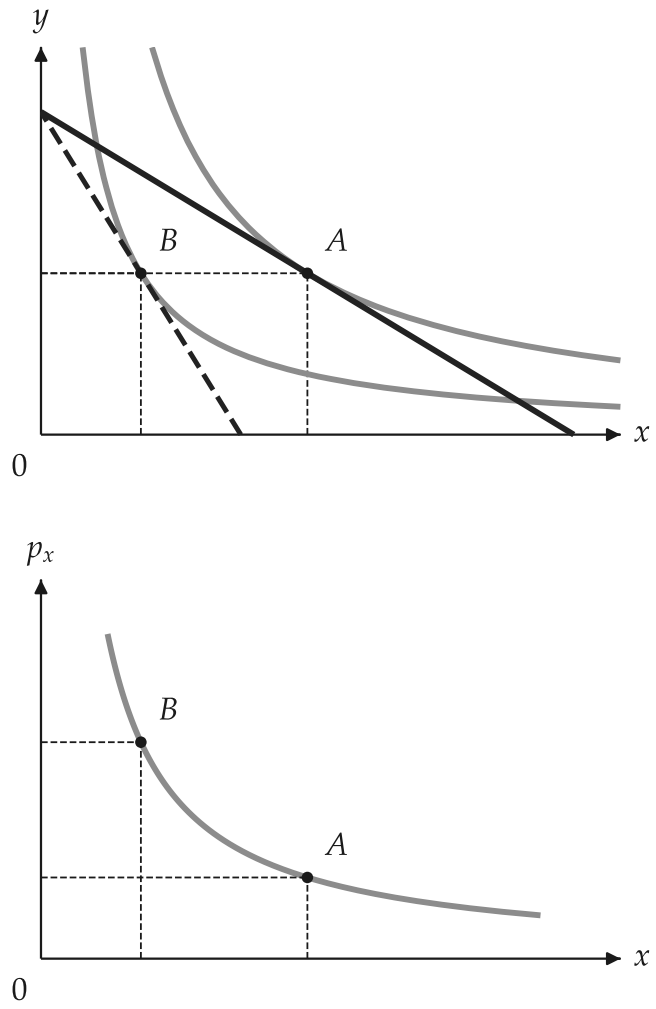



**图 12**
需求曲线最优点。

新增：*v1.5.0*　将价格效应分解为替代效应与收入效应。Hicks 补偿 (Hicks, 1939) 维持原效用水平；Slutsky 补偿 (Slutsky, 1915) 则使原消费组合在新价格下恰好可负担。返回值包含下列字段：

`A, B, C`　原始、补偿后与最终的消费组合。

`substitution_effect`　替代效应。

`income_effect`　收入效应。

`total_effect`　总效应。

`compensated_income`　补偿后的收入。

`DecompositionMethod`　`DecompositionMethod.HICKS, SLUTSKY`

新增：*v1.5.0*　决定补偿组合 B 的方式。

`HICKS`　保持原来的效用水平：B 在原来的无差异曲线上。

*DecompositionMethod*

`SLUTSKY`　让原来的组合仍买得起：B 是新价格下通过 A 的预算线上的最优组合。

*DecompositionMethod*

`Canvas.add_decomposition`　在画布上标示 A、B、C 三个消费组合、预算线、无差异曲线与效应箭头。Hicks 分解绘制通过 A 与 C 的曲线；Slutsky 分解另绘制通过 B 的曲线。

新增：*v1.5.0*

更新：*v1.9.0*

`show_curves`　绘制分解用的无差异曲线；已有自定义无差异曲线时设为 `False`，避免重复。

*bool* · 默认 `True`

`curve_stroke`　无差异曲线的样式。

*Stroke* | *None* · 默认 `None`

`curve_label`　显示并设置 $U_0$、$U_1$ 与 $U_B$ 标签。

*Label* | *None* · 默认 `None`

`substitution, income`　替代效应与收入效应的样式（参见下方 `Effect`）。

*Effect* | *None* · 默认 `None`

`point_marker`　消费组合的标记。

*Marker* | *None* · 默认 `None`

`point_label`　消费组合的文字。

*Label* | *None* · 默认 `None`

`legend`　图例位置与样式（参见下方 `Legend`）。

*Legend* | *None* · 默认 `None`

`Effect`　`Effect(color=None, y=None, label=None, label_position="right", label_offset=4, opacity=None)`

新增：*v1.8.0*　设置替代效应或收入效应的颜色、横轴下方箭头高度、标签及透明度。线条本身仍可由

更新：*v1.9.0*　`substitution_stroke` 或 `income_stroke` 覆盖。

`Legend`　`Legend(position="auto", fontsize=None, frame=None, columns=None, visible=None, opacity=None)`

新增：*v1.9.0*　`"auto"` 选择与图形重叠最少的内侧角落；四个角落均与图形重叠时，图例移到右侧。也可指定 `"upper left"` 等内侧角落，或 `"top"`、`"bottom"`、`"left"`、`"right"` 等外侧位置。`Legend(visible=False)` 隐藏图例。

**例 13**

```
from utility_viz import (
    Canvas,
    DecompositionMethod,
    Effect,
    Label,
    Legend,
    decompose_price_effect,
)
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
result = decompose_price_effect(
    model,
    px=(2.0, 4.0),
    py=3.0,
    income=60.0,
    method=DecompositionMethod.HICKS,
)

(
    Canvas(x_max=25, y_max=25, title="Hicks decomposition")
    .add_decomposition(
        result,
        show_arrows=True,
        label_effects=True,
        show_x_projections=True,
        substitution=Effect(label="SE"),
        income=Effect(label="IE", opacity=0.8),
        curve_label=Label(position="right"),
        legend=Legend(position="bottom"),
    )
    .save("hicks.png")
)
```

下图同样由 MosaicKit 图层画成，两个效果的名称标在坐标轴下方。

## 5.5　Edgeworth 箱形图

`EdgeworthBox`　`EdgeworthBox(⟨效用函数 A⟩, ⟨效用函数 B⟩, total_x=⟨浮点数⟩, total_y=⟨浮点数⟩, ...)`

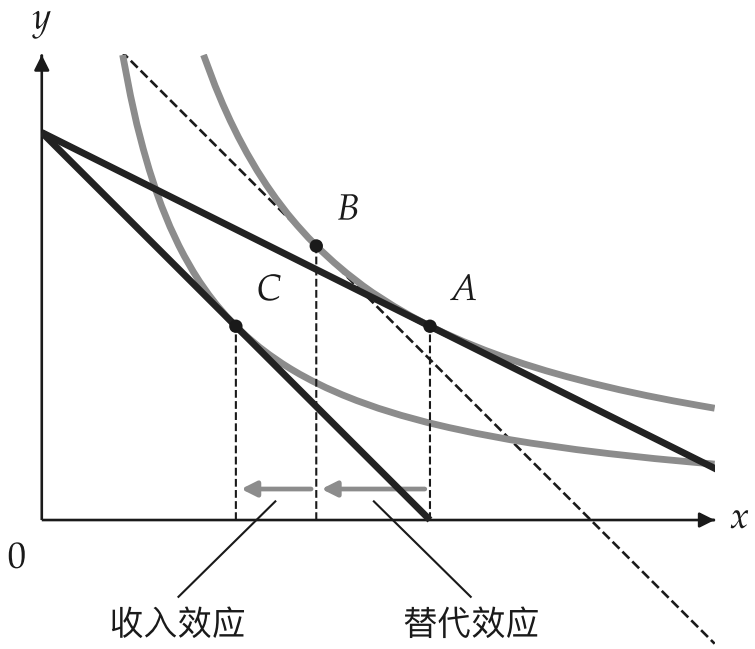



**图 13**　Hicks 分解。

新增：*v1.3.0*

创建两人交换经济的 Edgeworth 箱形图 (Edgeworth, 1881)。$A$ 的原点在左下，$B$ 的原点在右上。使用 `add_*` 方法添加禀赋、契约曲线、核、价格线与 Walras 均衡。

平滑偏好的契约曲线使用 `method="mrs"`；有折点、角解或自定义分段效用函数时，使用 `method="pareto"`。

**例 14**

```
from utility_viz import EdgeworthBox
from utility_viz.models import CobbDouglas

box = EdgeworthBox(
    CobbDouglas(alpha=0.8, beta=0.2),
    CobbDouglas(alpha=0.2, beta=0.8),
    total_x=12.0,
    total_y=10.0,
    title="Asymmetric Cobb-Douglas",
)

(
    box.add_endowment(5.0, 4.0)
    .add_contract_curve(n=100, method="mrs")
    .add_core()
    .add_price_line(px=1.2, py=1.0)
    .add_walrasian_equilibrium(px=1.2, py=1.0)
    .show_legend(loc="center left", bbox_to_anchor=(1.02, 0.5))
    .save("edgeworth.png")
)
```

下图同样由 MosaicKit 图层画成，以教科书的图为蓝本，画出两人的无差异曲线。

# 第 6 节　核心模型

内置模型位于 `utility_viz.models`，可传入 `solve()` 与 `Canvas.add_utility()`。本章按偏好特性分组，列出各模型的效用函数、参数、特有行为与图形。

## 6.1　平滑偏好

Cobb-Douglas、CES 与 Translog 的无差异曲线为平滑曲线；最优解取决于模型参数、价格与收入。

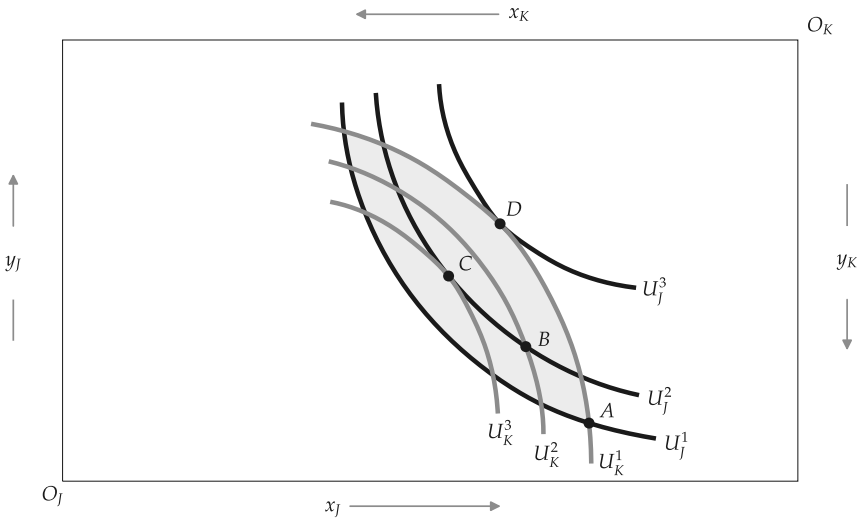



**图 14**
两位消费者通过配置 A 至 D 的无差异曲线；阴影透镜是双方都偏好于 A 的配置。

### Cobb-Douglas

`CobbDouglas` `CobbDouglas(alpha=0.5, beta=0.5)`

Cobb-Douglas 效用函数 (Cobb & Douglas, 1928)：

$$u(x,y) = x^{\alpha} y^{\beta}$$

正指数决定两种商品的相对权重，无差异曲线以坐标轴为渐近线。

`alpha` 商品 $x$ 的权重。

默认 `0.5`

`beta` 商品 $y$ 的权重。

默认 `0.5`

### CES

`CES` `CES(alpha=0.5, beta=0.5, rho=0.5)`

CES (Arrow et al., 1961) 通过 `rho` 调整商品间的替代弹性：

$$u(x,y) = (\alpha x^{\rho} + \beta y^{\rho})^{\frac{1}{\rho}}.$$

替代弹性为 $\sigma = 1/(1-\rho)$。极限形式分别为 Cobb-Douglas（$\rho \to 0$）、完全互补（$\rho \to -\infty$）与完全替代（$\rho \to 1$）。

`alpha` 商品 $x$ 的权重。

默认 `0.5`

`beta` 商品 $y$ 的权重。

默认 `0.5`

`rho` 替代参数，且 $\rho \neq 1$。

默认 `0.5`

**例 15**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CES

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CES(alpha=0.5, beta=0.5, rho=-0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("ces.png")
```

### Translog

`Translog` `Translog(alpha_0=0.0, alpha_x=0.5, alpha_y=0.5, beta_xx=0.0, beta_yy=0.0, beta_xy=0.0)`

新增：*v1.1.0*

Translog (Christensen et al., 1975) 以对数的一次项、二次项与交叉项表示效用：

$$\ln u(x,y) = \alpha_0 + \alpha_x \ln x + \alpha_y \ln y + \frac{1}{2}\beta_{xx}(\ln x)^2 + \frac{1}{2}\beta_{yy}(\ln y)^2 + \beta_{xy} \ln x \ln y.$$

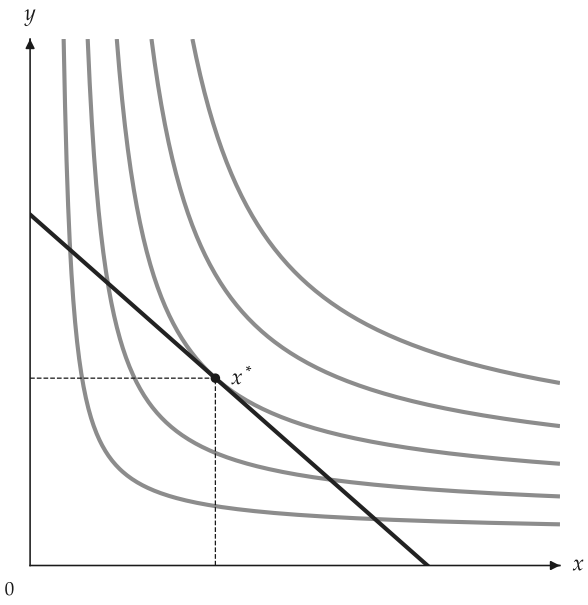



**图 15**
CES 曲线。

二次项与交叉项控制曲率。所有 $\beta$ 系数为零时，退化为 Cobb-Douglas 型的对数线性形式。

`alpha_0` 对数效用截距。

默认 `0.0`

`alpha_x` $\ln x$ 的一次项权重。

默认 `0.5`

`alpha_y` $\ln y$ 的一次项权重。

默认 `0.5`

`beta_xx` $x$ 方向的曲率。

默认 `0.0`

`beta_yy` $y$ 方向的曲率。

默认 `0.0`

`beta_xy` $x$ 与 $y$ 的交叉效应。

默认 `0.0`

**例 16**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import Translog

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = Translog(alpha_x=0.6, alpha_y=0.4, beta_xy=0.12)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=18, y_max=12, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=4))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("translog.png")
```

## 6.2　折点与角解

无差异曲线有折点或为直线时，最优解可能位于折点或坐标轴，不能仅以相切条件判定。

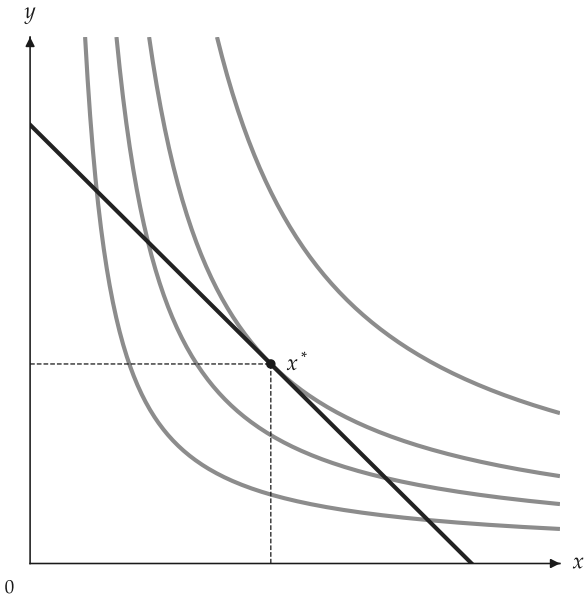



**图 16**
Translog 示例。

### 完全互补

`Leontief`　`Leontief(a=1.0, b=1.0)`

完全互补效用函数：

$$u(x, y) = \min\{ax, by\}$$

无差异曲线呈 L 型。权重与价格为正时，最优点满足固定比例 $ax = by$。

转角在图片与 TikZ 输出中都画成精确的直角；以 `CustomUtility` 创建的分段 `min/max` 函数亦同。

`a`　商品 $x$ 的权重。

默认 `1.0`

`b`　商品 $y$ 的权重。

默认 `1.0`

**例 17**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import Leontief

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = Leontief(a=1.0, b=1.0)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(
    model,
    levels=levels.around(eq.utility, n=5),
    show_rays=True,
    show_kinks=True,  # 每个折点画射线与标记
)
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("leontief.png")
```

### 完全替代

`PerfectSubstitutes`　`PerfectSubstitutes(a=1.0, b=1.0)`

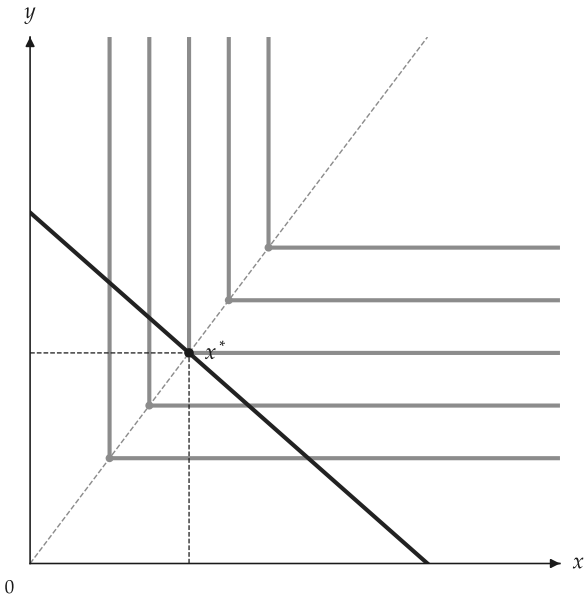



**图 17**
完全互补射线。

完全替代效用函数：

$$u(x, y) = ax + by.$$

无差异曲线为直线。每元边际效用不同时，最优解为仅购买较高者的角解；两者相等时，预算线上的消费组合皆为最优解。

a　商品 $x$ 的边际效用。

默认 `1.0`

b　商品 $y$ 的边际效用。

默认 `1.0`

**例 18**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import PerfectSubstitutes

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = PerfectSubstitutes(a=1.0, b=2.0)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("perfect_substitutes.png")
```

### 最大值效用

maximum　`CustomUtility(func=lambda x, y: np.maximum(a * x, b * y))`

以两个加权数量的最大值定义效用：

$$u(x, y) = \max\{ax, by\}$$

此偏好不具凸性。在正权重与正价格下，最优解位于效用更高的预算线轴截距。此模型以 `CustomUtility` 创建（详见第 7 节）。

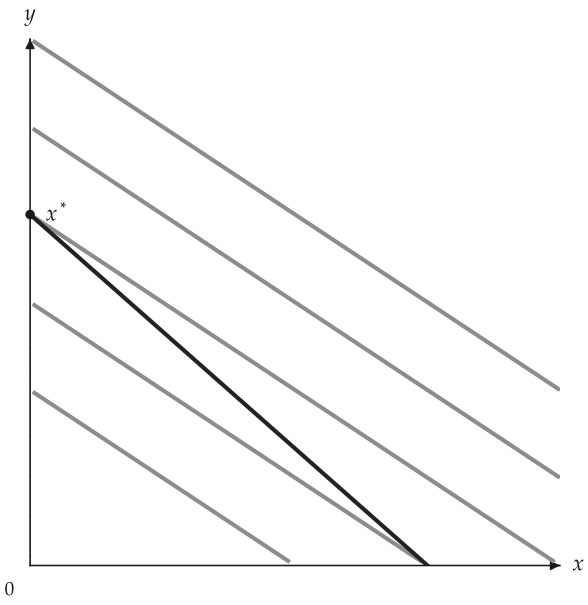



**图 18**
完全替代角解。

a　商品 $x$ 的权重。

默认 `1.0`

b　商品 $y$ 的权重。

默认 `1.0`

**例 19**

```python
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CustomUtility

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CustomUtility(func=lambda x, y: np.maximum(x, y), name="maximum")
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=25, y_max=20, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("maximum.png")
```

## 6.3　收入与参考点

以下模型分别描述准线性偏好、最低消费需求与具有饱和点的饱和偏好。

### 准线性

`QuasiLinear`　`QuasiLinear(v_func=numpy.log, linear_in="y")`
准线性效用中，一种商品以线性项表示：

$$u(x, y) = f(x) + y.$$

在内部解范围内，非线性商品的需求不受收入影响。`linear_in` 指定线性项对应的商品。

`v_func`　递增且凹的函数 $f$。

默认 `numpy.log`

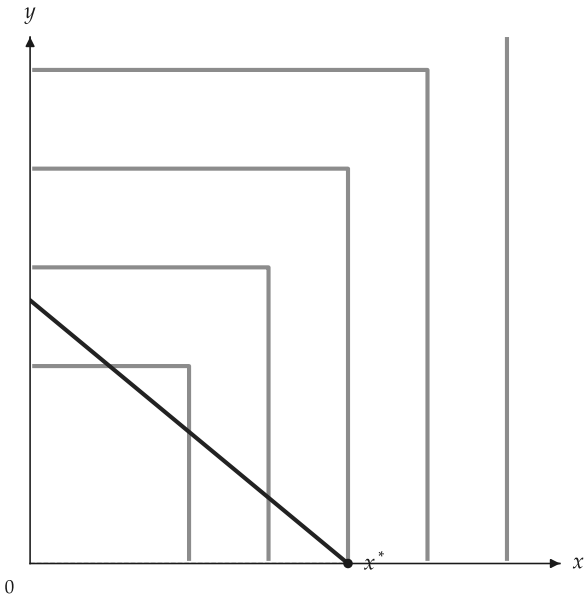



**图 19**
最大值效用。

`linear_in`　以线性方式进入的商品。

默认 `"y"`

**例 20**

```python
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import QuasiLinear

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = QuasiLinear(v_func=np.log, linear_in="y")
eq = solve(model, px=1.0, py=2.0, income=20.0)

cvs = Canvas(x_max=15, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(1.0, 2.0, 20.0)
cvs.add_equilibrium(eq)
cvs.save("quasi_linear.png")
```

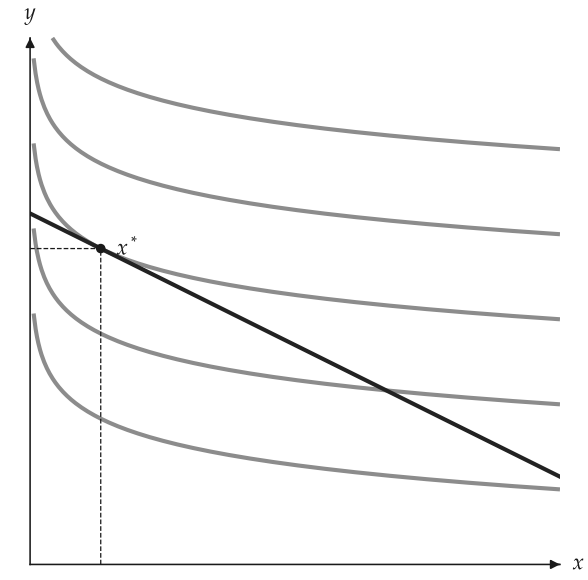



**图 20**
准线性偏好。

### Haagsma

`Haagsma` `Haagsma(alpha_x=1.0, alpha_y=2.0, gamma_x=2.0, gamma_y=27.0)`

新增：*v1.9.0*

Haagsma (2012) 效用函数为

$$u(x,y) = \alpha_x \ln(x-\gamma_x) - \alpha_y \ln\big(\gamma_y - y\big).$$

定义域为 $x > \gamma_x$ 且 $0 \le y < \gamma_y$，并要求 $0 < \alpha_x < \alpha_y$。商品 $x$ 的 Marshall 需求随收入下降；当 $\gamma_y p_y < I < \gamma_y p_y + \gamma_x p_x$ 时，商品 $x$ 为季芬财。

收入达到 $\gamma_y p_y + \gamma_x p_x$ 时，效用在预算线上没有有限最大值，模型会抛出 `InvalidParameterError`。

`alpha_x` 商品 $x$ 的权重，必须小于 `alpha_y`。

默认 `1.0`

`alpha_y` 商品 $y$ 的权重。

默认 `2.0`

`gamma_x` 商品 $x$ 的数量下界。

默认 `2.0`

`gamma_y` 商品 $y$ 的数量上界。

默认 `27.0`

`demand(px, py, income)` 返回闭式内部解。

`is_giffen(px, py, income)` 检查指定预算下商品 $x$ 是否为季芬财。

**例 21**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import Haagsma

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = Haagsma(gamma_y=10.0)
eq = solve(model, px=1.0, py=1.0, income=11.0)

cvs = Canvas(x_max=9, y_max=12, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=4))
cvs.add_budget(1.0, 1.0, 11.0)
cvs.add_equilibrium(eq)
cvs.save("haagsma.png")
```

### Stone-Geary

`StoneGeary` `StoneGeary(alpha=0.5, beta=0.5, bar_x=1.0, bar_y=1.0)`

Stone-Geary (Geary, 1950; Stone, 1954) 在 Cobb-Douglas 中添加最低消费量：

$$u(x,y) = (x-\bar{x})^{\alpha}(y-\bar{y})^{\beta}.$$

满足基本需求量 $\bar{x}$ 与 $\bar{y}$ 后，剩余收入依权重分配。

`alpha` 超额 $x$ 的权重。

默认 `0.5`

`beta` 超额 $y$ 的权重。

默认 `0.5`

`bar_x` 基本需求量 $\bar{x}$。

默认 `1.0`

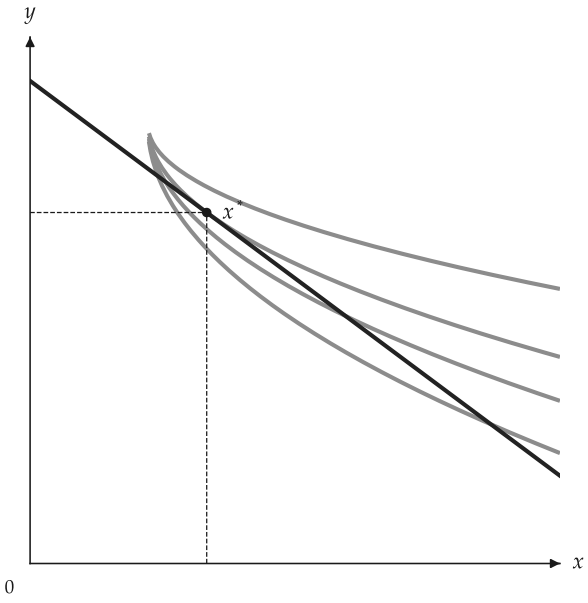



**图 21**
$\gamma_y = 10$ 的 Haagsma。

`bar_y` 基本需求量 $\bar{y}$。

默认 `1.0`

**例 22**

```python
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import StoneGeary

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = StoneGeary(alpha=0.5, beta=0.5, bar_x=2.0, bar_y=2.0)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("stone_geary.png")
```

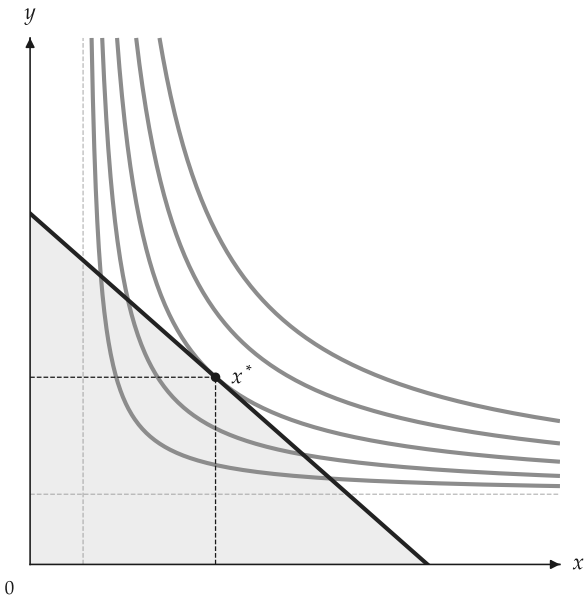



**图 22**
Stone-Geary。

### 饱和偏好

`Satiation`　`Satiation(bliss_x=5.0, bliss_y=5.0, a=1.0, b=1.0)`

饱和偏好以 $(x^*, y^*)$ 为效用最高的饱和点：

$$u(x,y) = -a(x-x^*)^2 - b(y-y^*)^2.$$

当 $a, b > 0$，偏离饱和点会使效用下降，无差异曲线为封闭椭圆，偏好不具单调性。

`bliss_x`　饱和点坐标 $x^*$。

默认 `5.0`

`bliss_y`　饱和点坐标 $y^*$。

默认 `5.0`

`a`　$x$ 轴方向的曲率。

默认 `1.0`

`b`　$y$ 轴方向的曲率。

默认 `1.0`

**例 23**

```python
import numpy as np

from utility_viz import Canvas, Config, levels
from utility_viz.models import Satiation

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = Satiation(bliss_x=6.0, bliss_y=4.0)

# 曲线取自效用值网格
X, Y = np.meshgrid(np.linspace(0.1, 12.0, 300), np.linspace(0.1, 10.0, 300))

cvs = Canvas(x_max=12, y_max=10, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.percentile(model(X, Y), n=5))  # u 的五个百分位数
cvs.save("satiation.png")
```

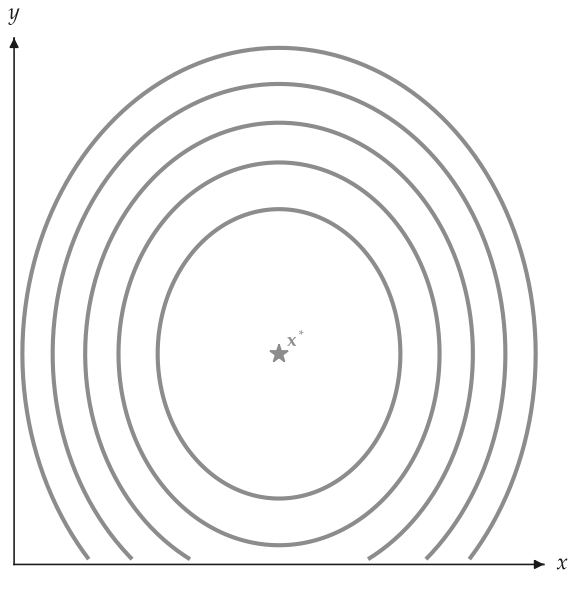



**图 23**
饱和偏好。

# 第 7 节　进阶模型

`CustomUtility` 定义二元效用函数。`MultiGoodCD` 表示多商品 Cobb-Douglas；固定其他商品的数量后，可将模型投影为二维图。

## 自定义效用函数

`CustomUtility`　`CustomUtility(func=⟨可调用对象⟩, name=⟨字符串⟩)`

接受向量化的 Python 函数，返回的模型可用于 `solve()` 与 `Canvas`。函数须接受两个 NumPy 数组，并返回相同形状的效用数组。以下以对数效用为例：

$$u(x, y) = \ln x + \ln y.$$

`func`　以 $x$ 与 $y$ 为变量的向量化效用函数。

`name`　模型的显示名称。

**例 24**

```python
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import CustomUtility

Config.load("manual.toml").use()  # 本手册图形所用的设置

# 任何数组函数都可作效用
model = CustomUtility(func=lambda x, y: np.log(x) + np.log(y), name="log+log")
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("custom.png")
```

## 多商品 Cobb-Douglas

`MultiGoodCD`　`MultiGoodCD(⟨权重⟩)`

`MultiGoodCD.freeze`　`.freeze(⟨商品⟩=⟨数量⟩, ...)`

`MultiGoodCD` 表示 $N$ 种商品的 Cobb-Douglas：

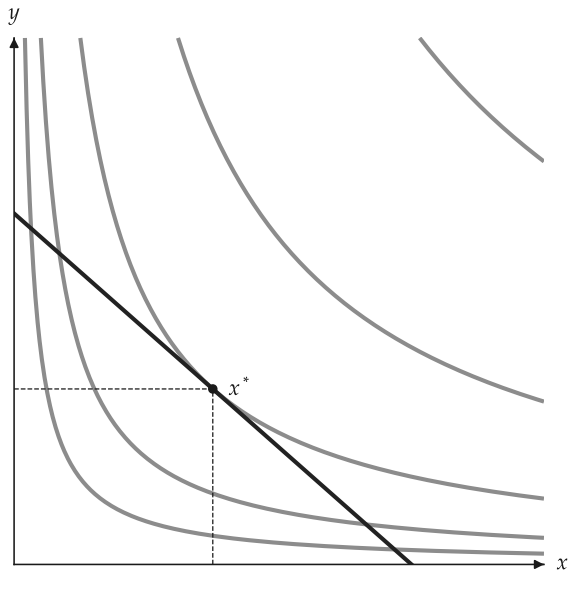



**图 24**
对数自定义模型。

$$u(x_1, \ldots, x_N) = \prod_{i=1}^{N} x_i^{\alpha_i}.$$

使用 `freeze()` 固定 $x$、$y$ 以外的商品数量，返回可用于二维 `Canvas` 的 `CustomUtility`。

`shares`　商品名称与指数 $\alpha_i$ 的对应。

`freeze(...)`　$x$、$y$ 以外商品的固定数量。

**例 25**

```
from utility_viz import Canvas, Config, levels, solve
from utility_viz.models import MultiGoodCD

Config.load("manual.toml").use()  # 本手册图形所用的设置

# 三种商品 固定 z = 10
model = MultiGoodCD({"x": 0.3, "y": 0.3, "z": 0.4})
two_good_model = model.freeze(z=10.0)
eq = solve(two_good_model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(two_good_model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("multigood.png")
```

### 与后端无关的场景

`utility_viz.core.scenes` 生成与 `Canvas` 相同的经济元素，但输出为不可变的 mosaickit 图层，而非直接绘制在 matplotlib 坐标轴上。每个图层都有固定的 id（`budget`、`budget.fill`、`equilibrium.drop`、`ic.2` 等）与以点分隔的主题角色（`utility.budget`、`utility.indifference.secondary` 等），因此场景可供检查、修改，并交由任何 mosaickit 后端渲染。这是额外添加的高级 API，`Canvas` 与其他图形不受影响。

`budget_layers`　`budget_layers(⟨浮点数⟩, ⟨浮点数⟩, ⟨浮点数⟩, fill=False, ...)`

`equilibrium_layers`　`equilibrium_layers(⟨均衡⟩, drop_lines=True, ...)`

`indifference_layers`　`indifference_layers(⟨模型⟩, ⟨效用水平⟩, ⟨上限⟩, ⟨上限⟩, ...)`

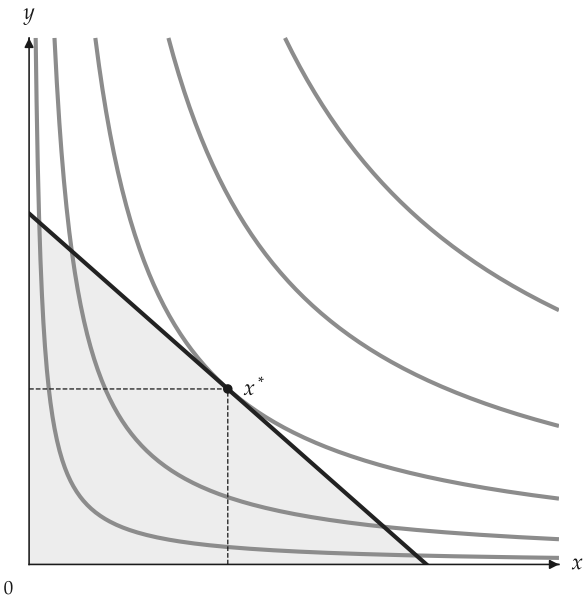



**图 25**
三商品固定 $z = 10$。

新增：*v2.0.0b1*

分别返回预算线 $p_x x + p_y y = I$、均衡消费组合，以及模型在 $[0, x_{\max}] \times [0, y_{\max}]$ 范围内无差异曲线的图层 tuple。样式来自主题角色，也可传入部分字段的 `Stroke`、`Marker` 与 `Fill` 覆盖。`indifference_layers` 返回 `(layers, levels)`，并以 bezierkit 将曲线描成 Bézier 路径，因此也能处理非单调与有折角的效用函数。

`UTILITY_THEME`

`UTILITY_THEME` 是含 `utility.*` 角色的 mosaickit 主题，配色与默认主题相同。

`canvas_to_tikz`

`to_tikz(canvas)` 将场景导出为 TikZ，曲线使用原生的 `.. controls ..` 语法。

新增：*v2.0.0b1*

```
from mosaickit import Canvas, CanvasSpec, quadrant_axes

from utility_viz.core.scenes import (
    UTILITY_THEME,
    budget_layers,
    canvas_to_tikz,
    equilibrium_layers,
    indifference_layers,
)
from utility_viz.models import CobbDouglas, solve

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)
u = eq.utility
ic, _ = indifference_layers(model, [0.6 * u, u, 1.4 * u], 18, 12)

spec = CanvasSpec(x_range=(0, 18), y_range=(0, 12))
canvas = (
    Canvas(spec, theme=UTILITY_THEME)
    .extend(quadrant_axes(18, 12))
    .extend(ic)
    .extend(budget_layers(2.0, 3.0, 30.0, fill=True))
    .extend(equilibrium_layers(eq))
)
canvas.save("scene.png")
tex = canvas_to_tikz(canvas)
```

TikZ 导出支持路径、填充、标记与文本图层。描绘的效用函数在绘图范围内必须为有限值。

# 第 8 节　分析

分析 API 提供 Marshall 需求的比较静态、Slutsky 矩阵，以及效用函数的齐次性与位似性检查。

## 8.1　比较静态

`comparative_statics`

`comparative_statics(⟨模型⟩, px=⟨浮点数⟩, py=⟨浮点数⟩, income=⟨浮点数⟩)`

新增：*v1.1.0*

以有限差分估计两商品 Marshall 需求对 $p_x$、$p_y$ 与收入的六个偏导数，返回 `Comparative Statics`。

- 在 `solve(...)` 的结果附近使用中央有限差分，默认的相对步长是 `1e-3`。
- 检测到自身价格导数为正或收入导数为负时会发出警告，分别对应季芬财与劣等财的需求特性。

`dx_dpx` $\partial x^*/\partial p_x$。
*float*

`dx_dpy` $\partial x^*/\partial p_y$。
*float*

`dx_dI` $\partial x^*/\partial I$。
*float*

`dy_dpx` $\partial y^*/\partial p_x$。
*float*

`dy_dpy` $\partial y^*/\partial p_y$。
*float*

`dy_dI` $\partial y^*/\partial I$。
*float*

```
from utility_viz import comparative_statics
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.4, beta=0.6)
cs = comparative_statics(model, px=2.0, py=3.0, income=60.0)

print(round(cs.dx_dpx, 1), round(cs.dx_dpy, 1), round(cs.dx_dI, 1))
print(round(cs.dy_dpx, 1), round(cs.dy_dpy, 1), round(cs.dy_dI, 1))

# -6.0 0.0 0.2
# 0.0 -4.0 0.2
```

## 8.2　Slutsky 矩阵

`slutsky_matrix`　`slutsky_matrix(⟨模型⟩, px=⟨浮点数⟩, py=⟨浮点数⟩, income=⟨浮点数⟩)`

更新：*v1.2.3*　以 Marshall 需求导数与收入效应计算两商品的 Slutsky 替代矩阵，并检查对称性、负半定性与齐次性；检查未通过时发出警告。使用 `as_array()` 可转为 NumPy 数组。

```
from utility_viz import slutsky_matrix
from utility_viz.models import CobbDouglas

S = slutsky_matrix(
    CobbDouglas(alpha=0.4, beta=0.6),
    px=2.0, py=3.0, income=60.0,
)

print(round(S.s_xx, 1), round(S.s_xy, 1))
print(round(S.s_yx, 1), round(S.s_yy, 1))
print(S.as_array().round(1))

# -3.6 2.4
# 2.4 -1.6
# [[-3.6  2.4]
#  [ 2.4 -1.6]]
```

## 8.3　齐次性

`HomogeneityAnalyzer`　`HomogeneityAnalyzer(⟨模型⟩)`

新增：*v1.1.0*　检查效用函数的齐次性、位似性与需求的零次齐次性。

`degree()`　估计齐次次数，详见第 8.4 节。

`euler_check(x, y)`　计算指定消费组合的 Euler 定理残差。

`is_homothetic()` 边际替代率（MRS）在等比例缩放下是否不变。

`demand_degree_zero(px, py, income)` Marshall 需求是否为零次齐次。

```
from utility_viz.models import CobbDouglas, HomogeneityAnalyzer

analyzer = HomogeneityAnalyzer(CobbDouglas(alpha=0.4, beta=0.6))
result = analyzer.degree()

print(round(result.degree, 6))
print(result.returns_to_scale)
print(round(analyzer.euler_check(3.0, 4.0), 6))
print(analyzer.is_homothetic())
print(analyzer.demand_degree_zero(px=2.0, py=3.0, income=60.0))

# 1.0
# ReturnsToScale.CONSTANT
# 0.0
# True
# True
```

## 8.4　规模报酬

`degree()` 返回 `HomogeneityResult`，包含下列字段：

`degree` 估计的齐次次数。

*float*

`is_homogeneous` 是否为齐次函数。

*bool*

`returns_to_scale` 规模报酬的分类，取值如下。

*ReturnsToScale*

Cobb-Douglas 的齐次次数为 $\alpha+\beta$：

$$u(\lambda x,\lambda y)=\lambda^{\alpha+\beta}u(x,y).$$

`ReturnsToScale` `ReturnsToScale.INCREASING`, `CONSTANT`, `DECREASING`, `NOT_HOMOGENEOUS` 函数的规模报酬分类。

`INCREASING` $\alpha+\beta>1$，规模报酬递增。

*ReturnsToScale*

`CONSTANT` $\alpha+\beta=1$，规模报酬不变。

*ReturnsToScale*

`DECREASING` $\alpha+\beta<1$，规模报酬递减。

*ReturnsToScale*

`NOT_HOMOGENEOUS` 没有一致的齐次次数。

*ReturnsToScale*

```
def shifted_utility(x, y):
    return x**0.4 * y**0.6 + 1.0


models = [
    CobbDouglas(alpha=0.7, beta=0.6),
    CobbDouglas(alpha=0.4, beta=0.6),
    CobbDouglas(alpha=0.2, beta=0.5),
    shifted_utility,
]
```

```
for model in models:
    result = HomogeneityAnalyzer(model).degree()
    degree = None if result.degree is None else round(result.degree, 1)
    print(degree, result.returns_to_scale.name)

# 1.3 INCREASING
# 1.0 CONSTANT
# 0.7 DECREASING
# None NOT_HOMOGENEOUS
```

# 第 9 节　主题与设置

Theme 定义图形的默认外观；Config 从 TOML 文件加载主题覆盖与字体设置。Canvas、Figure、DemandDiagram、价格效应分解及 EdgeworthBox 共用这些样式对象。

## 9.1　内置主题

themes

```
from utility_viz import Canvas, themes

cvs = Canvas(theme=themes.paper)
```

更新：*v1.11.0*

Python API 提供下列主题。主题对象不可变；需要调整字段时，可创建新的 Theme，或通过 Config 覆盖指定字段。

**表 6**
内置主题

| 名称 | 特性 |
|---|---|
| default | 色盲友好的默认配色 |
| colorblind | 与 default 相同的配色别名 |
| nord | 采用 Nord 色盘的冷色系主题 |
| paper | 透明背景与出版用细线条 |
| monochrome | 以线型与标记区分图层的黑白主题 |
| presentation | 放大标签、线条与标记，供投影使用 |
| dark | 深色背景与相应的前景色 |

Python API 与配置文件的 base 可使用全部七个名称[5]。

默认主题的颜色取自一组色盲友好的配色 (thriveth, 2014)。

themes.COLORBLIND_CYCLE_HEX 与 themes.COLORBLIND_CYCLE_RGB 提供这组配色的十六进制与 RGB 值。经济学教学大量依赖图形，因此不应只靠颜色传达含义，以免视觉障碍的学生无法辨识 (Kugler & Andrews, 1996)。

下列各图用内置主题（dark 除外）画出同一张 Cobb-Douglas 图，线宽沿用本手册，底色为白色。

[5]在命令行中直接传入 --theme 时，只接受 colorblind 别名以外的六个主题。

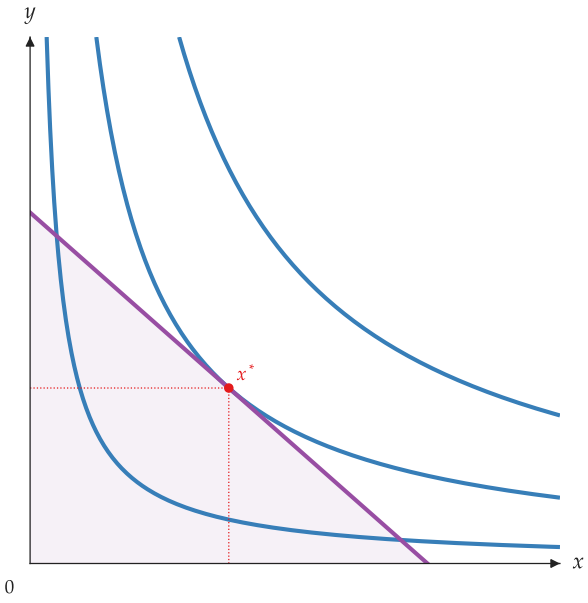



**图 26**　default 主题。

**例 26**

```python
from dataclasses import replace

from utility_viz import Canvas, levels, solve, themes
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

# 主题配色 本手册线宽
theme = replace(
    themes.default,
    background_color="#ffffff",
    ic_linewidth=2.6,
    budget_linewidth=2.6,
    eq_markersize=5.0,
)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$", theme=theme)
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("theme_default.png")
```

**例 27**

```python
from dataclasses import replace

from utility_viz import Canvas, levels, solve, themes
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

# 主题配色 本手册线宽
theme = replace(
    themes.nord,
    background_color="#ffffff",
    ic_linewidth=2.6,
    budget_linewidth=2.6,
    eq_markersize=5.0,
)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$", theme=theme)
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("theme_nord.png")
```

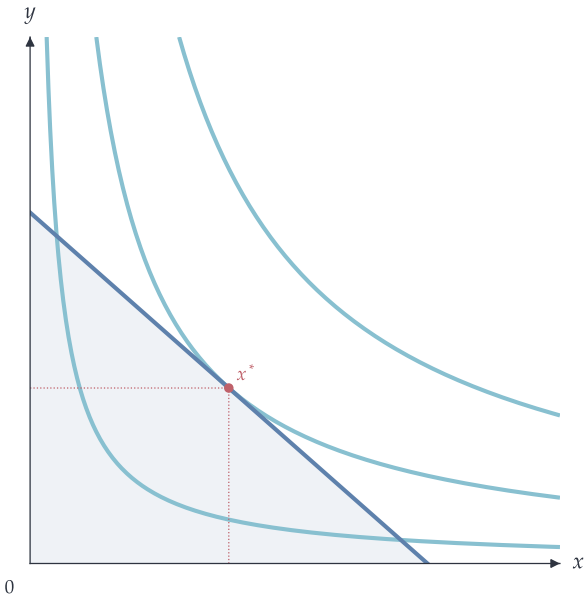



**图 27** nord 主题。

**例 28**

```python
from dataclasses import replace

from utility_viz import Canvas, levels, solve, themes
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

# 主题配色 本手册线宽
theme = replace(
    themes.paper,
    background_color="#ffffff",
    ic_linewidth=2.6,
    budget_linewidth=2.6,
    eq_markersize=5.0,
)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$", theme=theme)
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("theme_paper.png")
```

**例 29**

```python
from dataclasses import replace

from utility_viz import Canvas, levels, solve, themes
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

# 主题配色 本手册线宽
theme = replace(
    themes.monochrome,
    background_color="#ffffff",
    ic_linewidth=2.6,
    budget_linewidth=2.6,
    eq_markersize=5.0,
)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$", theme=theme)
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("theme_monochrome.png")
```

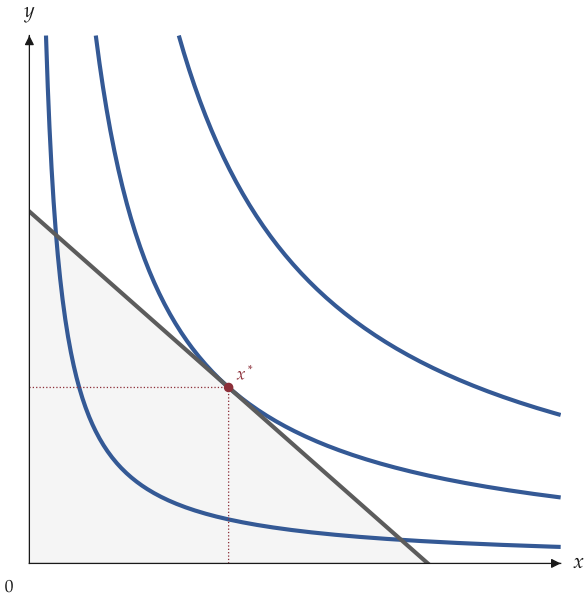



**图 28**
paper 主题。

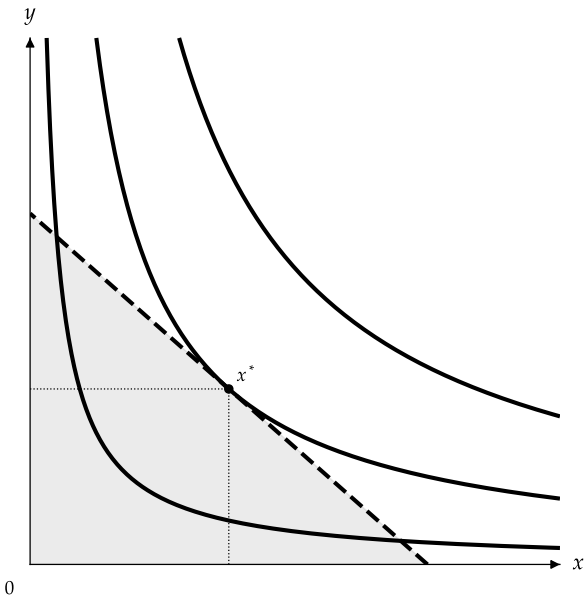



**图 29**
monochrome 主题。

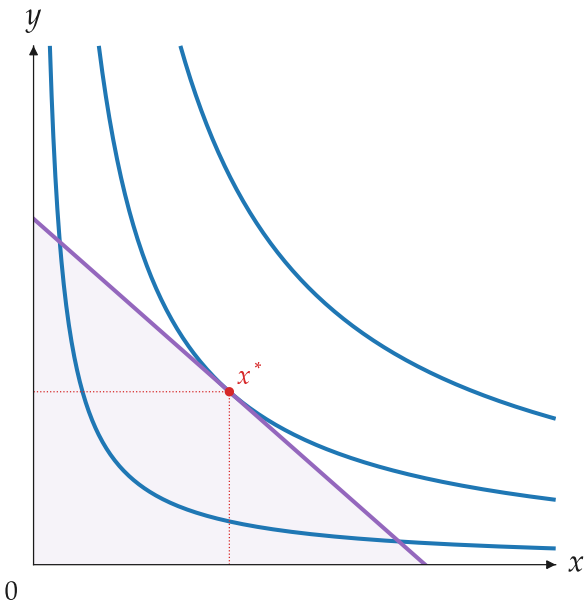



**图 30**　presentation 主题。

**例 30**

```python
from dataclasses import replace

from utility_viz import Canvas, levels, solve, themes
from utility_viz.models import CobbDouglas

model = CobbDouglas(alpha=0.5, beta=0.5)
eq = solve(model, px=2.0, py=3.0, income=30.0)

# 主题配色 本手册线宽
theme = replace(
    themes.presentation,
    background_color="#ffffff",
    ic_linewidth=2.6,
    budget_linewidth=2.6,
    eq_markersize=5.0,
)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$", theme=theme)
cvs.add_utility(model, levels=levels.around(eq.utility, n=3))
cvs.add_budget(2.0, 3.0, 30.0, fill=True)
cvs.add_equilibrium(eq)
cvs.save("theme_presentation.png")
```

## 9.2　自定义主题

Theme

```python
from utility_viz import Theme

my_theme = Theme(
    name="custom",
    background_color="white",
    label_scale=1.0,
    axis_color="#333333",
    label_color="#333333",
    ic_color="#2563eb",
    secondary_ic_color="#93c5fd",
    secondary_ic_opacity=0.45,
    budget_color="#dc2626",
    eq_color="#16a34a",
)
```

更新：*v1.12.0*

Theme 保存图形元素的颜色、线宽与样式默认值。未指定的字段使用类默认值；绘图方法明确收到的样式参数优先于主题。

`name`
主题标识名称。

*str*
`background_color`
图形与坐标区域的背景色；`None` 保持透明。

*str* | *None* · 默认 `None`
`label_scale`
普通标签字号的倍数。

*float* · 默认 `1.0`
`axis_color`
坐标轴颜色。

*str* · 默认 `"#222222"`
`label_color`
坐标标签与原点文字颜色。

*str* · 默认 `"#222222"`
`ic_color`
无差异曲线颜色。

*str* · 默认 `"#377EB8"`
`ic_linewidth`
无差异曲线线宽。

*float* · 默认 `1.8`
`secondary_ic_color`
非焦点无差异曲线颜色；`None` 沿用 `ic_color`。

*str* | *None* · 默认 `None`
`secondary_ic_linewidth`
非焦点无差异曲线线宽。

*float* · 默认 `1.0`
`secondary_ic_opacity`
非焦点无差异曲线不透明度。

*float* · 默认 `0.45`
`path_color`
PCC 与 ICC 路径颜色。

*str* · 默认 `"#4DAF4A"`
`budget_color`
预算线颜色。

*str* · 默认 `"#984EA3"`
`budget_fill_alpha`
预算集阴影不透明度。

*float* · 默认 `0.08`
`eq_color`
均衡点与投影线颜色。

*str* · 默认 `"#E41A1C"`
`eq_markersize`
均衡点标记大小。

*float* · 默认 `4.0`

`Theme` 也包含射线、折点、补偿预算线与价格效应箭头的颜色和线宽。`axis_stroke`、`drop_stroke`、`projection_stroke`、`guide_stroke` 与 `box_stroke` 保存其他线条的默认值；标记、标签、填色及图例则由 `Marker`、`Label`、`Fill` 与 `Legend` 属性提供（详见第 4.3 节）。

## 9.3　配置文件

`Config.load`
`Config.use`
`Config.reset`

```
from utility_viz import Config

Config.load("utility-viz.toml").use()
# 此后创建的图形使用该设置

Config.reset()
# 恢复内置默认值
```

新增：*v1.10.0*　`Config.load()` 读取 TOML 文件并返回 `Config`。`use()` 将其设为后续图形的默认配置；`reset()` 恢复内置默认值。

**教程：统一样式**

同一份讲义或论文的图形，通常要共用相同的颜色与线条。与其在每个 `Canvas` 重复传入参数，不如把样式写在项目根目录的 `utility-viz.toml`，让 Python 与命令行读取同一份配置。

1. **创建模板**。在项目根目录运行下列命令，会生成含注释的 `utility-viz.toml`，列出每个配置节可用的名称与字段：

   ```
   utility-viz init
   ```

   模板中的设置默认都是注释；只保留需要修改的项目即可，其余可以删除。

2. **选择基础主题**。`base` 指定起始的内置主题（表 6），其余设置都在它之上覆盖：

   ```
   base = "paper"
   ```

3. **覆盖颜色与样式**。只写出要改的字段：

   ```
   [color]
   ic = "#1B4F72"
   eq = "#C0392B"

   [stroke.budget]
   width = 1.6
   style = "dashed"

   [marker.eq]
   size = 5

   [label.point]
   fontsize = 11
   position = "right"
   ```

   配置节名称对应主题属性：`[color]` 的 `ic` 对应 `ic_color`，`[stroke.budget]` 对应 `theme.budget_stroke`；配置节内的字段就是 `Stroke`、`Marker`、`Label`、`Fill`、`Legend` 的参数（详见第 4.3 节）。字体写在 `[font]` 的 `text` 与 `math`。省略的字段保留基础主题的值。

4. **在 Python 中应用**。在创建任何图形之前加载一次：

   ```
   from utility_viz import Config

   Config.load().use()    # 默认读 ./utility-viz.toml
   ```

   此后创建的 `Canvas`、`Figure`、`DemandDiagram` 与 `EdgeworthBox` 都使用这组配置；调用 `Config.reset()` 可恢复内置默认值。

5. **在命令行应用**。`utility-viz plot` 会自动读取当前目录的 `utility-viz.toml`；要使用其他文件时，以 `--config` 指定配置文件：

   ```
   utility-viz plot --config utility-viz.toml --model cobb-douglas \
     --px 2 --py 3 --income 30 --output figure.png
   ```

配置的优先级由高到低为：直接传入 `Canvas` 或绘图方法的参数、配置文件、基础主题。例如 `Canvas(theme=themes.default)` 会略过配置文件中的主题。

配置文件写错时，`Config.load()` 会抛出 `InvalidParameterError`，并列出可用的名称。例如把 `[stroke.budget]` 误写成 `[stroke.budgt]`：

```
utility-viz.toml: [stroke.budgt]: no such setting (choose: axis, box,
budget, ...)
```

**手册图形设置**

本手册的示例都先加载 `manual.toml`，图形的外观由它决定：灰色曲线、黑色坐标轴与辅助线、黑色的点、白色底。文件没有指定字体，在任何电脑上都能运行；图形用 TeX Gyre Pagella 排版，没有这个字体时用默认字体。

```
# 本手册所有图形的设置
# 灰色曲线 黑色轴 白底
# 画图前先加载
#
#     Config.load("manual.toml").use()
#
# 各区段对应 Theme 属性

base = "default"

# 字体是 TeX Gyre Pagella 但不指定
# 要用请加上 [font] text = "TeX Gyre Pagella"

[color]
background = "#ffffff"
axis = "#1a1a1a"
label = "#1a1a1a"
ic = "#8c8c8c"                # 无差异曲线
secondary_ic = "#b8b8b8"      # 强调曲线旁的曲线
path = "#1a1a1a"              # 价格与收入消费路径
budget = "#222222"
eq = "#1a1a1a"                # 均衡点与其他点
ray = "#8c8c8c"
kink = "#8c8c8c"
sub_effect = "#8c8c8c"
inc_effect = "#8c8c8c"
compensated_budget = "#1a1a1a"
subsistence = "#8c8c8c"
contract = "#8c8c8c"
core = "#1a1a1a"
price = "#8c8c8c"
walrasian = "#1a1a1a"

[stroke.axis]
width = 1.0
style = "solid"
arrow = "-|>"

[stroke.ic]
width = 2.6

[stroke.secondary_ic]
width = 1.8
opacity = 1.0

[stroke.path]
width = 1.4

[stroke.budget]
width = 2.6

[stroke.ray]
```

```toml
width = 0.9

[stroke.substitution]
width = 1.8

[stroke.income]
width = 1.8

[stroke.compensated_budget]
width = 1.0

[stroke.contract]
width = 2.6

[stroke.price]
width = 2.6

[stroke.drop]                # 从点到坐标轴的辅助线
width = 0.8
style = "dashed"
color = "#1a1a1a"

[stroke.projection]
width = 0.8
style = "dashed"
color = "#1a1a1a"

[stroke.guide]
width = 0.8
style = "dashed"
color = "#1a1a1a"

[marker.eq]
size = 5

[marker.point]
size = 5

[label.point]                # 点旁的文字 如 x*
fontsize = 14
position = "right"
offset = 10

[fill.budget]
opacity = 0.08
```

**配置文件查找顺序**

`Config.discover`　`Config.discover([⟨路径⟩], directory=None)`

新增：*v2.0.0b1*　查找并读取配置文件；找不到时不会出错。

`Config.load()`、`Config.discover()` 与 `utility-viz plot` 按下列顺序确定配置文件：

1. 明确指定的路径（`Config.load("file.toml")` 或 `--config`）：文件必须存在，且只读取该文件；
2. 当前目录（或 `directory=`）中的 `utility-viz.toml`；
3. 旧的 `econ-viz.toml`，并发出弃用警告；
4. 内置默认值。

两个文件同时存在时，以 `utility-viz.toml` 为准，旧文件会被忽略并发出警告。两个函数只在找不到文件时有差异：`Config.load()` 与 1.x 相同，抛出 `InvalidParameterError`（"config file not found"）；`Config.discover()` 则返回内置默认值。`utility-viz init --migrate` 可将旧文件转为 `utility-viz.toml`（详见第 11.3 节）。

# 第 10 节　导出

## 10.1　输出格式

Canvas.save　`cvs.save(⟨路径⟩)`

根据 `path` 的扩展名决定输出格式。`Figure`、`DemandDiagram` 与 `EdgeworthBox` 使用相同接口。

- PNG：位图，分辨率由画布的 `dpi` 设置，默认为 300。
- PDF、SVG：向量图，可在打印或缩放时保留线条质量。
- TeX：TikZ 源代码（`.tex`），可嵌入 LaTeX 文件。通过关键字参数 `tikz_scale` 与 `tikz_standalone` 调整输出。

```
cvs.save("figure.png")   # PNG 默认 300 DPI
cvs.save("figure.pdf")   # PDF 向量
cvs.save("figure.svg")   # SVG 向量
cvs.save("figure.tex")   # TikZ
```

`dpi` 仅影响位图。预览时可降低分辨率：

```
cvs = Canvas(x_max=20, y_max=15, dpi=150)   # 降低 DPI 加快预览
```

## 10.2　GIF 动画

使用 `Animator.save()` 导出 GIF，完整示例详见第 12 节。

```
from utility_viz.core.animation import Animator

Animator(draw_frame, frames=frames).save("animation.gif", fps=12,
dpi=120)
```

## 10.3　交互窗口

`cvs.show()` 打开 matplotlib 交互窗口，不会保存文件。命令行的 `plot` 省略 `--output` 时也会打开窗口（详见第 11.2 节）：

```
utility-viz plot --model cobb-douglas --px 2 --py 3 --income 30
```

# 第 11 节　命令行工具

命令行工具可绘图、批量处理及输出需求公式。以 `uv tool install utility-viz` 安装为独立工具，或在 uv 项目中于命令前加上 `uv run`（详见第 2.5 节）。由 econ-viz 兼容包安装的 1.x 命令 `econ-viz` 在 2.x 仍可使用：它会发出弃用警告，再以相同参数运行 `utility-viz`（详见第 2.8 节）。

表 7
命令行命令

| 命令 | 说明 |
|---|---|
| `utility-viz help [⟨命令⟩]` | 显示命令行工具或特定命令的说明 |
| `utility-viz models` | 列出所有支持的效用模型 |
| `utility-viz plot ...` | 生成并导出图形 |
| `utility-viz solve-tex ...` | 以 TeX 格式输出 Marshall 需求的闭式解 |
| `utility-viz init [path]` | 创建 `utility-viz.toml` 配置模板 |

## 11.1　说明与模型

utility-viz help

`utility-viz help [⟨命令⟩]`

未指定参数时列出所有命令；指定命令名称时显示该命令的选项。

```
utility-viz help          # 所有命令
utility-viz help plot     # plot 选项
utility-viz help models   # models 选项
```

utility-viz models

列出命令行接口支持的模型名称与参数。`--model` 使用 kebab-case 名称，与 Python 类名称不同（参见表 8）。参数选项详见第 11.2 节。

表 8
命令行模型

| 模型 | `--model` 值 | 必要参数 | 可选参数 |
|---|---|---|---|
| Cobb-Douglas | `cobb-douglas` | `alpha、beta` | – |
| CES | `ces` | `rho` | `alpha、beta` |
| 完全互补 | `leontief` | `a、b` | – |
| 完全替代 | `perfect-substitutes` | `a、b` | – |
| 饱和偏好 | `satiation` | `bliss-x、bliss-y` | `a、b` |
| 准线性 | `quasi-linear` | – | `v-func、linear-in` |
| Stone-Geary | `stone-geary` | – | `alpha、beta、bar-x、bar-y` |
| Translog | `translog` | – | `alpha-0、alpha-x、alpha-y、beta-xx、beta-yy、beta-xy` |

## 11.2　绘图

utility-viz plot

`utility-viz plot --model ⟨名称⟩⟨选项⟩`

`utility-viz plot --latex ⟨算式⟩⟨选项⟩`

使用 `--model` 指定模型，或使用 `--latex` 输入效用函数，两者择一。指定 `--output` 时导出文件，省略时打开交互窗口。

预算线与均衡点需要完整的价格与收入。`--no-budget`、`--no-equilibrium`、`--no-curves` 可分别省略对应图层，且可并用（参见表 9）。

表 9
图层控制选项

| 情况 | 结果 |
|---|---|
| 提供完整 `--px`、`--py`、`--income` | 绘制预算线并求解均衡 |
| 省略任一价格或收入 | 仅绘制无差异曲线，不画预算线与均衡点 |
| 加上 `--no-equilibrium` | 跳过求解与均衡标注，即使提供了价格与收入 |
| 加上 `--no-budget` | 隐藏预算线，即使提供了价格与收入 |
| `--no-curves` 搭配完整价格与收入 | 只呈现预算线与均衡点，不画无差异曲线 |

```
# 指定模型名称
utility-viz plot --model cobb-douglas --alpha 0.5 --beta 0.5 ...

# LaTeX 表达式
utility-viz plot --latex "x^{0.4} y^{0.6}" ...
```

**例 31**

```
# Cobb-Douglas，可行集合加阴影
utility-viz plot --model cobb-douglas --alpha 0.5 --beta 0.5 \
              --px 2 --py 3 --income 30 \
              --fill --output cobb_douglas.png

# LaTeX 输入，Nord 主题，扩张路径
utility-viz plot --latex "x^{0.4} y^{0.6}" \
              --px 2 --py 3 --income 30 \
              --theme nord --show-ray \
              --output cd_latex.png

# 完全互补，加大画布
utility-viz plot --model leontief --a 1 --b 2 \
              --px 2 --py 3 --income 30 \
              --x-max 20 --y-max 15 \
              --output leontief.png

# CES，只画无差异曲线
utility-viz plot --model ces --rho -0.5 \
              --x-max 20 --y-max 15 --n-curves 6 \
              --no-budget --no-equilibrium \
              --output ces.png

# 饱和（饱和点）
utility-viz plot --model satiation --bliss-x 6 --bliss-y 4 \
              --x-max 12 --y-max 10 \
              --no-budget --no-equilibrium \
              --output satiation.png

# 不加 --output：打开窗口
utility-viz plot --model cobb-douglas --px 2 --py 3 --income 30
```

**选择模型**

`--model`，`-m`　模型名称，完整列表可用 `utility-viz models` 查询。

`--latex`，`-l`　LaTeX 表达式，支持 Cobb-Douglas、完全互补、完全替代与 CES（详见第 14 节）。

**价格与收入**

`--px`　商品 $x$ 的价格。

`--py`　商品 $y$ 的价格。

`--income`　消费者的收入。

**模型参数**

`--alpha`　alpha 参数（Cobb-Douglas、CES）。

默认 `0.5`

`--beta` beta 参数（Cobb-Douglas、CES）。

默认 `0.5`

`--a` 参数 $a$（完全互补、完全替代、饱和）。

默认 `1.0`

`--b` 参数 $b$（完全互补、完全替代、饱和）。

默认 `1.0`

`--rho` 替代参数（CES）。

默认 `0.5`

`--bliss-x` 饱和点的 $x$ 坐标（饱和）。

默认 `5.0`

`--bliss-y` 饱和点的 $y$ 坐标（饱和）。

默认 `5.0`

准线性、Stone-Geary 与 Translog 另有下列选项，可按模型使用：

`--v-func` 准线性的非线性函数，接受 `log` 或 `sqrt`。

默认 `log`

`--linear-in` 准线性的线性商品，接受 `x` 或 `y`。

默认 `y`

`--bar-x` Stone-Geary 的最低消费量 $\bar{x}$。

默认 `1.0`

`--bar-y` Stone-Geary 的最低消费量 $\bar{y}$。

默认 `1.0`

`--alpha-0` Translog 的截距。

默认 `0.0`

`--alpha-x` Translog 中 $\ln x$ 的一次项权重。

默认 `0.5`

`--alpha-y` Translog 中 $\ln y$ 的一次项权重。

默认 `0.5`

`--beta-xx` Translog 中 $x$ 方向的曲率。

默认 `0.0`

`--beta-yy` Translog 中 $y$ 方向的曲率。

默认 `0.0`

`--beta-xy` Translog 中 $x$ 与 $y$ 的交叉项。

默认 `0.0`

```
utility-viz plot --model stone-geary --bar-x 2 --bar-y 2 \
                 --px 2 --py 3 --income 30 \
                 --x-max 20 --y-max 15 --output stone_geary.png
```

此例的最低消费支出为 $2 \times 2 + 3 \times 2 = 10$，收入 30 高于此限制。若将收入改为 10 或更低，求解会因不符合模型定义域而失败。

**画布与图层**

`--x-max` 横轴上限。

默认 `10`

`--y-max` 纵轴上限。

默认 `10`

`--x-label` 横轴标签。

默认 `x`

`--y-label` 纵轴标签。

默认 `y`

`--title` 图形标题。

`--theme` 主题名称：`default`、`nord`、`paper`、`monochrome`、`presentation` 或 `dark`（详见第 9 节）。

默认 `default`

`--config` TOML 配置文件路径。命令行参数的优先级高于配置文件。省略时，若当前目录有 `utility-viz.toml`（或旧的 `econ-viz.toml`）则自动使用（详见第 9.3.3 节）。

`--n-curves` 无差异曲线的条数。

默认 `5`

`--dpi` 位图输出分辨率。

默认 `300`

`--fill` 为预算线下方的可行集合加上阴影。

`--show-ray` 画出通过最优点的扩张路径射线。

`--no-budget` 不画预算线。

`--no-equilibrium` 不画均衡点。

`--no-curves` 不画无差异曲线。

`--output, -o` 输出文件（`.png`、`.pdf`、`.svg`、`.tex`）；省略时打开交互窗口。
批量生成图形时，为每组参数指定不同文件名，并先创建输出目录。`--x-max` 与 `--y-max` 不会随价格自动调整；轴截距或最优点超出范围时，应同步增大画布上限。

## 11.3　创建配置文件

`utility-viz init` `utility-viz init [⟨路径⟩] [--force] [--migrate]`

新增：*v1.10.0*

创建带注释的 `utility-viz.toml` 模板。省略路径时写入当前目录；文件已存在时，必须使用 `--force` 才会覆盖。`--migrate` 则改为从同一目录中旧的 `econ-viz.toml` 创建 `utility-viz.toml`，并保留旧文件。

```
utility-viz init
utility-viz init config/figures.toml
utility-viz init --migrate
utility-viz plot --config utility-viz.toml --model cobb-douglas \
             --px 2 --py 3 --income 30 --output figure.png
```

## 11.4　需求公式

`utility-viz solve-tex` `utility-viz solve-tex --model ⟨名称⟩⟨选项⟩`

以 TeX 格式输出 Marshall 需求的闭式解，可用于支持 TeX 数学式的文件。

```
# 数值参数
utility-viz solve-tex --model cobb-douglas --alpha 0.4 --beta 0.6

# 符号参数
utility-viz solve-tex --model cobb-douglas --symbolic-params

# 自定义价格与收入符号
utility-viz solve-tex --model leontief --a 2 --b 3 \
                  --px-symbol p_1 --py-symbol p_2 --income-symbol M
```

支持 cobb-douglas、leontief、perfect-substitutes 及其 LaTeX 表达式。此命令只输出公式文字，不生成图像文件。

`--symbolic-params` 保留模型参数符号，例如 $\alpha$、$\beta$；未使用时代入指定的参数值。

`--px-symbol` 商品 $x$ 价格的符号。

默认 `p_x`

`--py-symbol` 商品 $y$ 价格的符号。

默认 `p_y`

`--income-symbol` 收入的符号。

默认 `I`

## 第 12 节　动画

`Animator` 根据指定的参数序列调用绘图函数，将返回的 `Canvas` 或 `Figure` 导出为 GIF。使用前需安装 `animation` 可选依赖（详见第 2.4 节）。从 `utility_viz.core.animation` 导入；1.x 为 `econ_viz.animation`。

`Animator` `Animator(⟨绘图函数⟩, frames=⟨数值⟩).save(⟨路径⟩, fps=12, dpi=120, loop=0)`

新增：*v1.4.0*

依次将 `frames` 的值传入 `draw`，每次返回的图形对应一个帧。

- `save()` 使用 Pillow，不需要 `ffmpeg`。
- 导出前将帧合成至白色背景，避免残影。
- `fps`、`dpi` 与 `loop` 分别设置帧率、分辨率与循环次数。

**例 32**

```python
import numpy as np

from utility_viz import Canvas, levels, solve
from utility_viz.core.animation import Animator
from utility_viz.models import CobbDouglas

def draw(px: float) -> Canvas:
    model = CobbDouglas(alpha=0.5, beta=0.5)
    eq = solve(model, px=px, py=2.0, income=20.0)
    lvls = levels.around(eq.utility, n=5)

    return (
        Canvas(
            x_max=14, y_max=12,
            x_label="X_1", y_label="X_2",
            title="Price sweep"
        )
        .add_utility(model, levels=lvls)
        .add_budget(px=px, py=2.0, income=20.0, fill=True)
        .add_equilibrium(eq, show_ray=True, drop_dashes=True)
    )

Animator(draw, frames=np.linspace(1.0, 6.0, 45)).save(
    "price_sweep.gif",
    fps=12,
    dpi=120,
)
```

`examples/scripts/animation.py` 提供四种示例[6]：

- 参数变动：固定价格与收入，调整效用函数参数。

[6] 各示例均包含 Cobb-Douglas、CES、完全替代与完全互补模型。

- 价格变动：固定效用函数与 $p_y$，调整 $p_x$，呈现预算线旋转，写法见上例。
- 收入变动：固定效用函数与价格，调整收入，呈现预算线平移。
- 预算线变动：仅显示预算线，省略效用图层。

四种示例的差异只在于 `draw()` 每次固定哪些值、调整哪个值。以参数变动为例，将上例的 `draw()` 改为接收 `alpha`，价格固定为 $p_x = 2$：

```
def draw(alpha: float) -> Canvas:
    model = CobbDouglas(alpha=alpha, beta=1.0 - alpha)
    eq = solve(model, px=2.0, py=2.0, income=20.0)
    ...

Animator(draw, frames=np.linspace(0.1, 0.9,
45)).save("parameter_sweep.gif")
```

收入变动将 `draw()` 的参数改为 `income`，并把该值传入 `add_budget()` 与 `solve()`。只呈现预算线时，省略 `add_utility()` 与 `add_equilibrium()`。

图 31 至图 33 各节录四个帧。

**例 33**

```
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.core.animation import Animator
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置


def draw(alpha: float) -> Canvas:
    """一格影格：x 的权重是 alpha，y 的权重是 1 - alpha。"""
    model = CobbDouglas(alpha=alpha, beta=1.0 - alpha)
    eq = solve(model, px=2.0, py=2.0, income=20.0)
    cvs = Canvas(x_max=14, y_max=12, x_label="$x_1$", y_label="$x_2$")
    cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
    cvs.add_budget(2.0, 2.0, 20.0, fill=True)
    cvs.add_equilibrium(eq, show_ray=True)
    return cvs


Animator(draw, frames=np.linspace(0.1, 0.9, 45)).save("parameter_sweep.gif", fps=12,
dpi=120)
```

下面每张图取 GIF 的四个帧，GIF 共 45 帧。

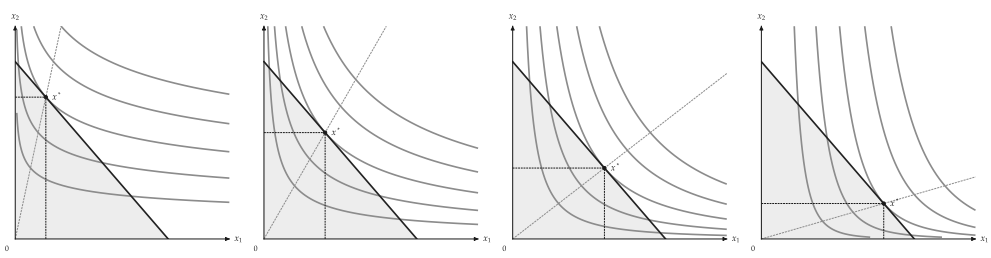


**图 31**
参数变动。

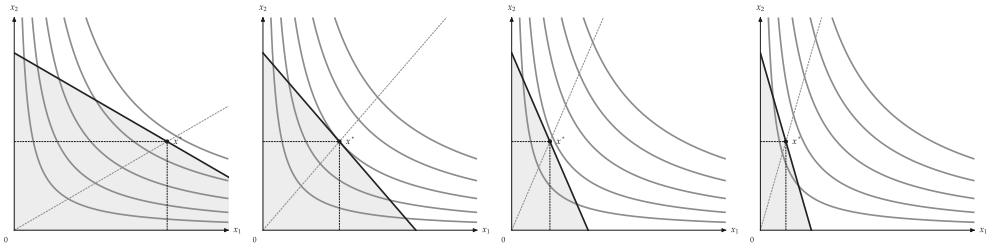


**图 32**
价格变动。

**例 34**

```python
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.core.animation import Animator
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
# 每帧用同一组曲线
fixed = levels.around(solve(model, px=2.0, py=2.0, income=20.0).utility, n=5)


def draw(px: float) -> Canvas:
    """一格影格：x 的价格是 px。"""
    eq = solve(model, px=px, py=2.0, income=20.0)
    cvs = Canvas(x_max=14, y_max=12, x_label="$x_1$", y_label="$x_2$")
    cvs.add_utility(model, levels=fixed)
    cvs.add_budget(px, 2.0, 20.0, fill=True)
    cvs.add_equilibrium(eq, show_ray=True)
    return cvs


Animator(draw, frames=np.linspace(1.0, 6.0, 45)).save("price_sweep.gif", fps=12,
dpi=120)
```

**例 35**

```python
import numpy as np

from utility_viz import Canvas, Config, levels, solve
from utility_viz.core.animation import Animator
from utility_viz.models import CobbDouglas

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = CobbDouglas(alpha=0.5, beta=0.5)
# 每帧用同一组曲线
fixed = levels.around(solve(model, px=2.0, py=2.0, income=20.0).utility, n=5)


def draw(income: float) -> Canvas:
    """一格影格：所得是 income。"""
    eq = solve(model, px=2.0, py=2.0, income=income)
    cvs = Canvas(x_max=14, y_max=12, x_label="$x_1$", y_label="$x_2$")
    cvs.add_utility(model, levels=fixed)
    cvs.add_budget(2.0, 2.0, income, fill=True)
    cvs.add_equilibrium(eq, show_ray=True)
    return cvs


Animator(draw, frames=np.linspace(8.0, 36.0, 45)).save("income_sweep.gif", fps=12,
dpi=120)
```

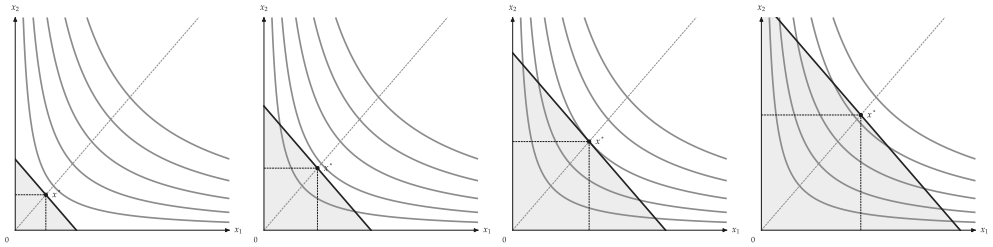


**图 33**
收入变动。

# 第 13 节　交互组件

`WidgetViewer` 在 Jupyter 中提供滑块与数值输入字段，调整参数后会更新图形。使用前需安装 `interactive` 可选依赖（详见第 2.4 节）。从 `utility_viz.core.interactive` 导入；1.x 为 `econ_viz.interactive`。

`WidgetViewer`
新增：*v1.4.0*

`WidgetViewer(⟨绘图函数⟩, ⟨名称⟩=(⟨下限⟩, ⟨上限⟩, ⟨间距⟩), ...).show()`
传入绘图函数 `draw`，并以（最小值，最大值，步长）指定各参数范围。每个参数会生成同步的 `FloatSlider` 与 `FloatText`；数值变更后重新调用 `draw`，并替换单元格中的旧图。`draw` 的写法与 `Animator` 相同（详见第 12 节），只是可同时接收多个参数。以第 12 节的价格变动示例为基础，让 `draw()` 另外接收 `alpha`：

**例 36**

```python
from utility_viz.core.interactive import WidgetViewer

def draw(alpha: float, px: float) -> Canvas:
    model = CobbDouglas(alpha=alpha, beta=1.0 - alpha)
    ...  # 其余与动画示例相同

WidgetViewer(
    draw,
    alpha=(0.2, 0.8, 0.05),
    px=(1.0, 6.0, 0.25),
).show()
```

## 13.1　笔记本设置

在新的 Jupyter 或 Colab 环境中，按下列顺序运行[7]：

1. 运行安装单元格。
2. 若安装过程升级 ipywidgets、traitlets 或 IPython，重新启动运行环境。
3. 重新启动后，从导入软件包的单元格继续运行，无须再次安装。

## 13.2　选择交互组件或 GIF

需要由用户调整参数时，使用 `WidgetViewer`；需要在演示文稿或网页中播放固定的变动过程时，使用 `Animator` 导出 GIF（详见第 12 节）。

[7] utility-viz 仓库中的 `notebook/econ-viz Playground.ipynb`（沿用旧文件名）已采用此流程，可在 Jupyter、VS Code 或 Colab 打开；环境中已有 utility-viz 时，会自动跳过安装。

# 第 14 节　LaTeX 解析

parse_latex　parse_latex(⟨算式⟩)

将 LaTeX 效用函数解析为模型，返回值可用于 solve()、Canvas 或其他接受模型的 API。

```
from utility_viz import parse_latex

model = parse_latex(r"x^{0.4} y^{0.6}")  # CobbDouglas(alpha=0.4,
beta=0.6)
model = parse_latex(r"\min(2x, 3y)")     # Leontief(a=2.0, b=3.0)
model = parse_latex(r"2x + 3y")          # PerfectSubstitutes(a=2.0,
b=3.0)
```

## 14.1　支持的形式

支持的输入形式参见表 10。省略的系数或指数默认为 1。

表 10
支持的效用函数。

| 类型 | 格式 | 示例 |
|---|---|---|
| Cobb-Douglas | `x^{α} y^{β}` 或 `x^α y^β` | `x^{0.3} y^{0.7}` |
| 完全互补 | `\min(ax, by)` 或 `min(ax, by)` | `\min(2x, y)` |
| 完全替代 | `ax + by` | `3x + 1.5y` |
| CES | `(αx^ρ + βy^ρ)^{1/ρ}` | `(0.4x^{-0.5}+0.6y^{-0.5})^{-2}` |

可省略开头的 `U =` 或 `u(x, y) =`：

```
U(x,y) = x^{0.5} y^{0.5}
U = x^{0.5} y^{0.5}
x^{0.5} y^{0.5}
```

**例 37**

```
from utility_viz import Canvas, Config, levels, parse_latex, solve

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = parse_latex(r"\min(2x, 3y)")  # Leontief(a=2.0, b=3.0)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5), show_kinks=True)
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("leontief.png")
```

**例 38**

```
from utility_viz import Canvas, Config, levels, parse_latex, solve

Config.load("manual.toml").use()  # 本手册图形所用的设置

model = parse_latex(r"2x + 3y")  # PerfectSubstitutes(a=2.0, b=3.0)
eq = solve(model, px=2.0, py=3.0, income=30.0)

cvs = Canvas(x_max=20, y_max=15, x_label="$x$", y_label="$y$")
cvs.add_utility(model, levels=levels.around(eq.utility, n=5))
cvs.add_budget(2.0, 3.0, 30.0)
cvs.add_equilibrium(eq)
cvs.save("perfect_substitutes.png")
```

## 14.2　错误处理

无法解析时会抛出 `utility_viz.ParseError`，错误信息包含支持的格式：

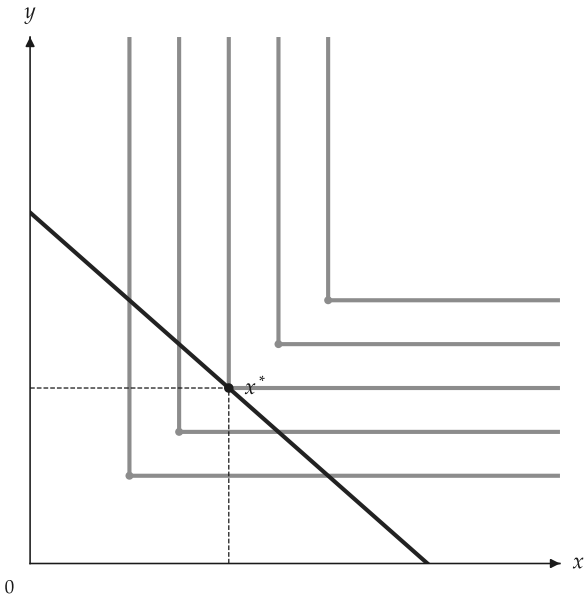



**图 34**
Leontief 示例。

```
from utility_viz import ParseError, parse_latex

try:
    model = parse_latex(r"x^2 + y^2")
except ParseError as e:
    print(e)

# Unrecognised LaTeX utility function: 'x^2 + y^2'
# Supported forms:
#   Cobb-Douglas       : x^{alpha} y^{beta}
#   Leontief           : \min(ax, by)
#   Perfect Substitutes: ax + by
#   CES                : (alpha x^{rho} + beta y^{rho})^{1/rho}
```

## 14.3　在命令行工具中使用

`utility-viz plot --latex` 使用相同的解析器，接受相同的输入格式（详见第 11.2 节）：

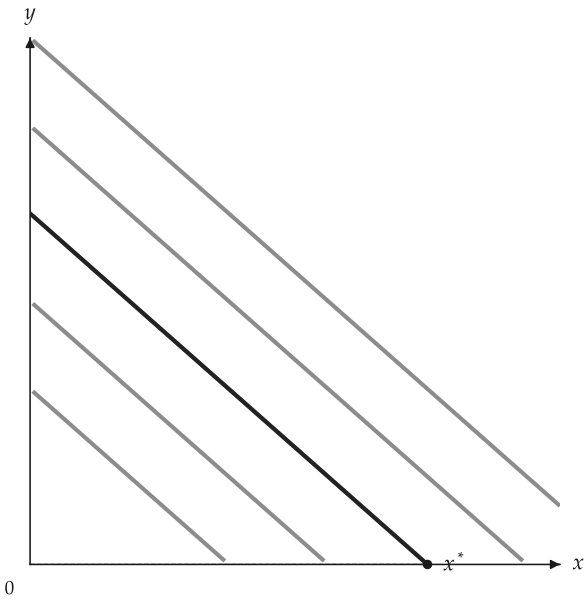



**图 35**
完全替代输入示例。

```
utility-viz plot --latex "x^{0.4} y^{0.6}" --px 2 --py 3 --income 30 -o
out.png
```

# 更新纪录

本章列出影响软件包使用的版本变更，不含仅修改文件的版本；各条目按 l3doc 惯例标注于它所描述的功能旁，页码即该功能的实际页码。

**v2.0.0b1** (2026-10-03)

**一般**：由 econ-viz 更名为 utility-viz：发行包、导入名称、命令与配置文件一并更名 ····· 2

内部结构重整为 `core` 与 `models`；公开 API 为软件包根目录与 `utility_viz.models` ····· 2

添加依赖 mosaickit 与 bezierkit ····· 2

整个仓库通过 `mypy` 检查；Ruff 限制行长、复杂度与语句数量 ····· 2

`utility-viz`：添加依赖 `mosaickit>=0.5.1,<0.6.0` 与 `bezierkit>=0.5.0rc1,<0.6.0` ··· 3

仓库改为 uv workspace，utility-viz 与 econ-viz 同步发布 ····· 3

命令改为 `utility-viz`；`econ-viz` 会打印弃用警告并转交给新命令 ····· 49

`econ-viz`：更名为 utility-viz；`econ_viz` 包、`econ-viz` 命令与 `econ-viz.toml` 查找保留为已弃用的兼容层，于 3.0.0 移除 ····· 5

兼容层改由独立的 econ-viz 发行包提供，升级 1.x 安装时不再删除与 utility-viz 共用的文件 ·· 5

`UtilityVizError`：`UtilityVizError` 成为基础异常类；`EconVizError` 保留为同一个类的别名 ····· 8

`Leontief`：折角效用函数的无差异曲线在 Matplotlib 与 TikZ 输出中保留精确直角 ····· 28

`core.scenes`：添加与后端无关的 mosaickit 图层工厂，涵盖预算线、均衡与无差异曲线，并可导出原生 Bézier TikZ ····· 36

`Config`：默认配置文件更名为 `utility-viz.toml`；旧的 `econ-viz.toml` 仍可读取，但会发出警告 ····· 45

`Config.discover`：添加 `Config.discover()`，找不到文件时改用内置默认值 ····· 45

`utility-viz plot`：未指定 `--config` 时，读取当前目录的 `utility-viz.toml`（或旧的 `econ-viz.toml`） ····· 49

`utility-viz init`：添加 `--migrate` 选项 ··· 49

`Animator`：移至 `utility_viz.core.animation` ····· 54

`WidgetViewer`：移至 `utility_viz.core.interactive` ····· 57

**v1.12.0** (2026-09-26)

**一般**：添加无差异曲线主次层级、焦点曲线及数值或序数标签 ····· 2

`Canvas.add_utility`：添加焦点效用层级、次要曲线样式及序数标签 ····· 8

`Theme`：添加次要无差异曲线的颜色、线宽与不透明度设置 ····· 40

**v1.11.0** (2026-09-26)

**一般**：完成主题系统，统一画布、多面板图、需求图、效应分解与 Edgeworth 箱形图的视觉设置 ····· 2

`Canvas`：背景、标签、线条与标记均由主题提供默认值 ····· 8

`Figure`：面板背景、标签、线条与标记完整应用主题 ····· 18

`EdgeworthBox`：箱框、契约曲线、核、禀赋、价格线与 Walras 均衡完整应用主题 ····· 24

`Theme`：完成主题系统，添加背景色、标签比例及所有经济图层的默认样式 ····· 40

`themes`：添加 `paper`、`monochrome`、`presentation` 与 `dark` ····· 40

`Config`：配置文件可选择所有内置主题，并在各种图形间保留相同的视觉语义 ····· 45

`econ-viz plot`：`--theme` 支持所有命令行内置主题 ····· 49

**v1.10.1** (2026-09-26)

**一般**：软件包添加 `py.typed`，并在 CI 运行 Ruff 与 Mypy 检查 ····· 2

**v1.10.0** (2026-09-26)

`econ-viz`：Python 3.10 使用 tomli 读取 TOML 配置文件 ····· 3

`Figure`：未指定主题与字体时，使用当前的 `Config` ····· 18

`EdgeworthBox`：未指定主题与字体时，使用当前的 `Config` ····· 24

`Config`：添加 `Config`、`econ-viz.toml`、`econ-viz init` 与 `plot --config` ····· 45

`econ-viz init`：添加配置文件模板命令与 `plot --config` ····· 49

**v1.9.0** (2026-09-26)

`Canvas`：坐标轴、原点、标题与其他文字元素接受 `Label` ····· 8

`Stroke`：添加 `opacity` 透明度 ····· 15

`DemandDiagram`：支持 `Legend`、`Marker`、`Label` 与各图层的 `Stroke` ····· 21

`Canvas.add_decomposition`：默认绘制 A、B、C 所在的无差异曲线，并支持曲线标签与 `Legend` ····· 21

横轴下方的效应箭头依 A→B、B→C 方向绘制；效应为零时不绘制箭头 ····· 21

`Haagsma`：添加具有闭式解的 Haagsma 效用模型，用于劣等财与季芬财分析 ····· 32

**v1.8.0** (2026-09-26)

`Canvas`：添加 `Axis`、`Marker`、`Label` 与 `Fill` 样式对象 ····· 8

`Stroke`：线条样式统一由 `Stroke` 表示；个别样式参数保留为简写 ····· 15

`Canvas.add_decomposition`：添加 `Effect`、`Marker` 与 `Label` 样式对象 ····· 21

`EdgeworthBox`：标记点、文字与坐标轴接受 `Marker`、`Label` 与 `Axis` ····· 24

**v1.7.0** (2026-09-25)

**一般**：CI 测试 Python 3.10–3.13，强制分支覆盖率，并对可运行的示例做冒烟测试 ····· 2

版本标签须通过测试后才会触发发布 ····· 2

`econ-viz`：软件包管理与构建改用 uv ····· 3

修正示例脚本，使其可在新检出的源代码目录中直接运行 ····· 4

添加 `econ-viz --version`，显示已安装的软件包版本 ····· 49

`Canvas`：支持为每幅图单独设置的文字与数学字体，不影响 matplotlib 的全局设置 ····· 8

`Canvas` 与 `Figure` 支持分别设置坐标轴标签位置、线条样式与箭头样式 ····· 15

`Stroke`：统一控制画布、多面板图、需求图与 Edgeworth 箱形图中线条的粗细、样式、颜色与箭头 ····· 15

`CobbDouglas`：修正效用模型的定义域、非对称 CES 的扩张路径、Cobb-Douglas 的极限情况，以及收入未花完时的饱和最优解 ····· 25

修正效用值接近零或为负时的等高线水平计算 · 25

`comparative_statics`：修正价格、收入接近零或基本需求限制时，比较静态计算超出定义域的问题 ····· 37

**v1.6.0** (2026-04-24)

**一般**：`examples/output/` 的输出文件不再纳入版本控制 ····· 2

`Canvas`：无差异曲线的默认线宽从 `2.0` 降为 `1.8` ····· 15

`themes.default`：默认配色改为色盲友好方案，并提供 `COLORBLIND_CYCLE_RGB` 与 `COLORBLIND_CYCLE_HEX` ····· 40

`Canvas.save`：TikZ 坐标轴的回归测试不再依赖颜色索引 ····· 49

`Animator`：动画示例移到 `examples/scripts/animation.py` ····· 54

**v1.5.0** (2026-04-21)

**一般**：添加效应分解、需求图、PCC/ICC 路径、多面板版面与 TikZ 导出的示例脚本 …… 2

添加 TikZ 导出与价格效应分解的回归测试 … 2

`econ-viz`：Colab 上的笔记本安装流程在重新启动后也能正常运作 …… 4

`Figure`：改善共享面板版面中坐标轴标签的位置与可见度 …… 18

`decompose_price_effect`：价格效应分解：`decompose_price_effect()`、`PriceEffect Decomposition` 与 `Canvas.add_decomposition()`，可画出 A/B/C 消费组合以及替代与收入效应 …… 21

效应分解的 API 可以直接从软件包根目录导入 · 21

`Canvas.save`：纯 TikZ 导出后端；`Canvas.save()` 与 `Figure.save()` 接受 `.tex` …… 49

**v1.4.0** (2026-04-09)

**一般**：添加 `Animator` 与 `WidgetViewer` 的测试 · 2

`econ-viz[extras]`：添加可选依赖 `animation`、`interactive` 与 `all` …… 4

`Animator.save`：导出前将帧合成至不透明背景，改善 GIF 导出稳定性 …… 49

`Animator`：`Animator` 以 Pillow 生成 GIF 参数变动动画，不需要 `ffmpeg` …… 54

添加效用参数、价格、收入与仅显示预算线的 GIF 示例 …… 54

`WidgetViewer`：`WidgetViewer` 在 Jupyter 笔记本中提供滑块控制 …… 57

笔记本交互组件添加数值输入字段 …… 57

**v1.3.2** (2026-04-03)

**一般**：发布流程跳过 PyPI 上已有的版本 …… 2

`EdgeworthBox`：Edgeworth 契约曲线与价格线默认为黑色虚线 …… 24

**v1.3.1** (2026-04-03)

**一般**：软件包根目录的导出改为延迟加载，公开 API 不变 …… 2

`Canvas`：Canvas 的绘制拆成 `canvas.renderers` 与 `canvas.primitives` …… 15

`EdgeworthBox`：Edgeworth 的内部实现拆成 compute、state 与 plotter 三个模块 …… 24

`models.registry`：添加模型注册表（`models.registry`），供命令行工具创建模型 …… 25

`CobbDouglas`：等高线水平的规则集中在 `contours.level_policies` …… 25

`Canvas.save`：Canvas、Figure 与 Edgeworth Box 共享同一个图形导出器（`io.exporter`） … 49

`econ-viz`：命令行错误改为抛出 `CliConfig Error`，结束处理集中在 `cli.main` …… 49

**v1.3.0** (2026-04-03)

**一般**：`examples/edgeworth_box.py` 涵盖常见的效用函数组合 …… 2

添加 Edgeworth 测试 …… 2

`EdgeworthBox`：添加 `EdgeworthBox` 与 `EquilibriumFocusConfig`，支持两人交换经济绘图 …… 24

契约曲线、核、Walras 均衡叠图，以及聚焦于均衡的无差异曲线 …… 24

**v1.2.3** (2026-03-31)

`slutsky_matrix`：`SlutskyMatrix` 会检查对称性、负半定性与齐次性，条件不成立时发出警告 …… 38

`econ-viz models`：命令行工具支持 `Quasi Linear`、`StoneGeary` 与 `Translog` …… 49

**v1.2.2** (2026-03-31)

**一般**：添加 PCC/ICC 示例生成器与测试 …… 2

`PricePath`：PCC/ICC 路径默认经过平滑处理，端点略微延长，除非指定否则不显示标记 … 20

PCC/ICC 路径改用独立配色，并增加需求图商品空间的留白 …… 20

**v1.2.0** (2026-03-30)

`Layout`：多面板 `Figure` 版面与 `Layout` 枚举 …· 19

`PricePath`：添加 `PricePath`、`IncomePath` 与 `Canvas.add_path()`，支持 PCC/ICC 绘图 ·· 20

`DemandDiagram`：添加 `DemandDiagram`，联动显示商品空间与 Marshall 需求 ····· 21

**v1.1.0** (2026-03-30)

`Translog`：`Translog` 模型，以及图例与无差异曲线标签 ····· 25

`comparative_statics`：`comparative_statics` 辅助函数 ····· 37

`HomogeneityAnalyzer`：分析模块中的 `HomogeneityAnalyzer` 与 `ReturnsToScale` ··· 38

**v1.0.2** (2026-03-29)

`econ-viz`：删除 NumPy 的版本上限，避免在 Colab 上发生冲突 ····· 3

`Canvas`：数学式的坐标轴标签不再被重复包装 · 15

**v1.0.1** (2026-03-29)

`econ-viz`：固定 `numpy<2`，避免 Colab 与本机安装时 ABI 不兼容 ····· 3

放宽 Python 与 NumPy 的版本限制；`pytest` 固定在 8.x ····· 3

**v1.0.0** (2026-03-28)

**一般**：首次发布 ····· 61

# 命令索引


**A**

add_budget ····· 10
add_equilibrium ····· 11
add_point ····· 13
add_ray ····· 13
add_utility ····· 9
Animator ····· 54
ArrowStyle ····· 16
Axis ····· 17

**B**

budget_layers ····· 36

**C**

Canvas ····· 7, 8
Canvas.add_* ····· 7
Canvas.add_decomposition ····· 22
Canvas.add_path ····· 20
Canvas.save ····· 8, 49
Canvas.show ····· 8
canvas_to_tikz ····· 37
CES ····· 26
CobbDouglas ····· 26
comparative_statics ····· 37
Config.discover ····· 48
Config.load ····· 45
Config.reset ····· 45
Config.use ····· 45
CustomUtility ····· 35

**D**

decompose_price_effect ····· 21
DecompositionMethod ····· 22
DemandDiagram ····· 21

**E**

EdgeworthBox ····· 24
Effect ····· 23
equilibrium_layers ····· 36

**F**

Figure ····· 18
Fill ····· 18

**H**

Haagsma ····· 32
HomogeneityAnalyzer ····· 38

**I**

IncomePath ····· 20
indifference_layers ····· 36

**L**

Label ····· 18
Layout ····· 19
Legend ····· 23
Leontief ····· 28
levels.around ····· 7
LineStyle ····· 15

**M**

Marker ····· 17
maximum ····· 29
MultiGoodCD ····· 35
MultiGoodCD.freeze ····· 35

**P**

parse_latex ····· 58
PerfectSubstitutes ····· 28
PricePath ····· 20

**Q**

QuasiLinear ····· 30

**R**

ReturnsToScale ····· 39

**S**

Satiation ····· 34
save ····· 14
show ····· 14
slutsky_matrix ····· 38
solve ····· 6

StoneGeary ····· 32
Stroke ····· 17

**T**

Theme ····· 44
themes ····· 40
Translog ····· 26

**U**

utility-viz help ····· 50
utility-viz init ····· 53
utility-viz models ····· 50
utility-viz plot ····· 50
utility-viz solve-tex ····· 53
UTILITY_THEME ····· 37
utility_viz.models ····· 6
UtilityVizDeprecationWarning ····· 5

**W**

WidgetViewer ····· 57

# 参考文献

Arrow, K. J., Chenery, H. B., Minhas, B. S., & Solow, R. M. (1961). Capital-labor substitution and economic efficiency. *Review of Economics and Statistics*, *43*(3), 225–250.

Christensen, L. R., Jorgenson, D. W., & Lau, L. J. (1975). Transcendental logarithmic utility functions. *American Economic Review*, *65*(3), 367–383.

Cobb, C. W., & Douglas, P. H. (1928). A theory of production. *American Economic Review*, *18*(1), 139–165.

Edgeworth, F. Y. (1881). *Mathematical psychics*. C. Kegan Paul.

Geary, R. C. (1950). A note on "A constant-utility index of the cost of living". *Review of Economic Studies*, *18*(1), 65–66.

Haagsma, R. (2012). A convenient utility function with Giffen behaviour. *ISRN Economics*, *2012*, Article 608645. https://doi.org/10.5402/2012/608645

Hicks, J. R. (1939). *Value and capital*. Clarendon Press.

Kugler, P., & Andrews, K. (1996). Graphical analysis and the visually impaired in undergraduate economics courses. *The Journal of Economic Education*, *27*(3), 224–228. https://doi.org/10.1080/00220485.1996.10844911

Slutsky, E. (1915). Sulla teoria del bilancio del consumatore. *Giornale degli Economisti e Rivista di Statistica*, *51*, 1–26.

Stone, R. (1954). Linear expenditure systems and demand analysis: An application to the pattern of British demand. *Economic Journal*, *64*(255), 511–527.

thriveth. (2014, January 22). *A color blind/friendly color cycle for Matplotlib line plots* [Source code]. https://gist.github.com/thriveth/8560036